



StockVelia

PRESENTADO POR:

Ivonne Dayana Sanchez Contreras

Jaider Andrés González Ovalle

Alan Felipe Caro Olaya

Jonathan Andres Jimenez Aguilera

N° FICHA: 3311976

PRESENTADO A:

Jeysson Contreras y Eduardo Foglia

ANÁLISIS Y DESARROLLO DE SOFTWARE

SENA - CENTRO DE GESTIÓN DE MERCADOS, LOGÍSTICA Y TECNOLOGÍAS DE LA
INFORMACIÓN (CGMLTI)

BOGOTÁ D.C

7 / 09 / 2026

Índice

1. Introducción
 - 1.1. Descripción del proyecto
 - 1.2. Contexto del proyecto
 - 1.3. Problema que resuelve
 - 1.4. Justificación
 - 1.5. Importancia
 - 1.6. Beneficios
 - 1.7. Público objetivo
 - 1.8. Alcance
 - 1.9. Limitaciones
2. Análisis del Problema
 - 2.1. Situación actual
 - 2.2. Problemática encontrada
 - 2.3. Causas
 - 2.4. Consecuencias
 - 2.5. Árbol de problemas
 - 2.6. Análisis de riesgos
 - 2.7. Necesidades identificadas
3. Objetivos
 - 3.1. Objetivo General
 - 3.2. Objetivo Específicos
 - 3.3. Metas
 - 3.4. Indicadores de éxito
4. Levantamiento de Requisitos
 - 4.1. Requisitos Funcionales
 - 4.2. Requisitos No Funcionales
 - 4.3. Requisitos de Negocio
5. Casos de Uso
 - 5.1. Actores del Sistema
 - 5.2. Matriz Resumen de Casos de Uso
 - 5.3. Especificación Detallada de Casos de Uso Principales
6. Historias de Usuario
 - 6.1. Gestión de Seguridad y Usuarios (HU-001 a HU-006)
 - 6.2. Gestión de Catálogo de Productos (HU-007 a HU-010)
 - 6.3. Movimientos e Inventario en Tiempo Real (HU-011 a HU-015)
 - 6.4. Reportes, Exportación y Búsqueda Avanzada (HU-016 a HU-020)
7. Diagramas UML
 - 7.1. Diagrama de Casos de Uso
 - 7.2. Diagrama de Clases
 - 7.3. Diagrama de Secuencia
 - 7.4. Diagrama de Actividades
 - 7.5. Diagrama de Estados
 - 7.6. Diagrama de Componentes
 - 7.7. Diagrama de Despliegue
 - 7.8. Diagrama de Paquetes

8. Arquitectura del Sistema
 - 8.1. Arquitectura General
 - 8.2. Arquitectura por Capas
 - 8.3. Arquitectura Elegida
9. Diseño de Bases de Datos
 - 9.1. Modelo Conceptual
 - 9.2. Modelo Lógico
 - 9.3. Modelo Físico
 - 9.4. Diagrama Entidad Relación
 - 9.5. Diccionario de Datos
10. Normalización de la Base de Datos
 - 10.1. Primera Forma Normal (1NF)
 - 10.2. Segunda Forma Normal (2NF)
 - 10.3. Tercera Forma Normal (3NF)
11. Diseño de Interfaces
 - 11.1. Módulo de Autenticación y Registro (Login / Register)
 - 11.2. Panel de Control Principal
 - 11.3. Gestión de Catalogo de Productos
 - 11.4. Registro de Movimientos de Inventario
12. Prototipos
13. Estructura del Proyecto
14. Tecnologías Utilizadas
15. APIs
 - 15.1. Método HTTP
 - 15.2. URL
 - 15.3. Descripción
 - 15.4. Parámetros
 - 15.5. Request
 - 15.6. Response
 - 15.7. Códigos de error
16. Seguridad
 - 16.1. Autenticación
 - 16.2. Autorización
 - 16.3. Roles
 - 16.4. Hash de contraseñas
 - 16.5. Protección SQL Injection
 - 16.6. Protección XSS
 - 16.7. Protección CSRF
 - 16.8. Validaciones backend
 - 16.9. Validaciones frontend
17. Rendimiento
 - 17.1. Optimización de consultas
 - 17.2. Índices
 - 17.3. Caché
 - 17.4. Consumo de memoria
 - 17.5. Consumo de CPU
18. Pruebas
 - 18.1. Plan de pruebas

- 18.2. Casos de prueba
- 18.3. Pruebas funcionales
- 18.4. Pruebas de integración
- 18.5. Pruebas de rendimiento
- 18.6. Pruebas de seguridad
- 18.7. Evidencias
- 18.8. Resultado esperado
- 18.9. Resultado obtenido
- 18.10. Estado
- 19. Gestión del Proyecto
 - 19.1. Metodología
 - 19.1.1. Scrum
 - 19.1.2. Kanban
 - 19.1.3. Cascada
 - 19.2. Cronograma
 - 19.3. Sprint
 - 19.4. Actividades
 - 19.5. Entregables
 - 19.6. Roles
- 20. Control de Versiones
 - 20.1. Estrategia Git
 - 20.2. Ramas
 - 20.3. Commits
 - 20.4. Releases
- 21. Instalación
 - 21.1. Requisitos previos
 - 21.2. Dependencias
 - 21.3. Configuración
 - 21.4. Variables de entorno
 - 21.5. Instalación
 - 21.6. Ejecución
 - 21.7. Despliegue
- 22. Manual Técnico
- 23. Manual de Usuario
- 24. Mantenimiento
- 25. Riesgos
- 26. Futuras Mejoras
- 27. Conclusiones
- 28. Recomendaciones
- 29. Bibliografía

1. Introducción

En el panorama empresarial actual, la supervivencia y el crecimiento de las microempresas y PYMES dependen de su agilidad financiera y de la precisión en la toma de decisiones. Tradicionalmente, la administración de mercancías se ha visto como una tarea operativa y secundaria, gestionada mediante métodos manuales o herramientas estáticas que no reflejan la realidad del negocio. Stock Velia trasciende el concepto tradicional de un software de control de inventarios para posicionarse como una plataforma de inteligencia financiera y blindaje de capital. Este documento detalla la conceptualización, arquitectura y alcance de una solución diseñada no solo para registrar entradas y salidas, sino para maximizar la rentabilidad, eliminar las fugas invisibles de dinero y transformar el stock estancado en liquidez inmediata.

1.1 Descripción del proyecto

Stock Velia es una solución tecnológica integral, compuesta por una aplicación web responsiva, desarrollada bajo un stack tecnológico moderno que incluye React en el frontend, Node.js con Express en el backend y PostgreSQL para la persistencia de datos relacionales con altos estándares de seguridad como autenticación por JWT y cifrado con Bcrypt.

A diferencia de los gestores convencionales, el sistema opera como un asistente financiero inteligente que monitorea el flujo de mercancía en tiempo real, automatiza alertas tempranas ante umbrales críticos de stock y centraliza la auditoría de movimientos. Su diseño prioriza una experiencia de usuario (UX/UI) intuitiva y ágil, permitiendo que cualquier comerciante, sin conocimientos técnicos avanzados, tome el control absoluto de los activos de su negocio desde cualquier dispositivo.

1.2 Contexto del proyecto

Las microempresas y PYMES en el mercado actual enfrentan un entorno altamente competitivo donde los márgenes de ganancia son estrechos. Gran parte de estos negocios operan bajo esquemas empíricos: registros en cuadernos, hojas de cálculo desconectadas o memoria operativa. Esta falta de estandarización genera puntos ciegos críticos en la cadena de suministro local.

Stock Velia surge como respuesta a la necesidad imperativa de digitalizar y optimizar los procesos comerciales de los pequeños negocios, alineándose con las exigencias del mercado actual de transformación digital y aprovechando metodologías ágiles de desarrollo de software (Scrum) para entregar un producto robusto, escalable y listo para operar en entornos reales de alta exigencia.

1.3 Problema que resuelve

El problema central que ataca Stock Velia no es la simple escasez de espacio para contar productos, sino el drenaje silencioso de capital operativo. Específicamente, resuelve:

1. **Fugas de dinero hormiga y mermas invisibles:** Pérdidas económicas por productos caducados, deterioro o diferencias de inventario no detectadas a tiempo.
2. **Quiebres de stock y pérdida de ventas:** La incapacidad de prever cuándo se agotará un producto de alta rotación, lo que se traduce en clientes insatisfechos que migran a la competencia.
3. **Dinero durmiente (Capital atrapado):** Exceso de mercancía retenida en bodega que no genera rotación, inmovilizando el flujo de caja que el negocio necesita para crecer.
4. **Cero visibilidad financiera:** Dependencia de registros manuales propensos a errores humanos, imposibilitando la obtención de reportes financieros y de auditoría confiables.

1.4 Justificación

Invertir tiempo y recursos en el desarrollo de Stock Velia se justifica bajo una premisa económica contundente: lo que no se mide con precisión, se pierde. Las soluciones existentes en el mercado o bien son demasiado complejas y costosas para una microempresa (como los grandes sistemas ERP), o son demasiado básicas (como un Excel) y no evitan los errores operativos.

Este proyecto demuestra la viabilidad técnica de integrar tecnologías web modernas para resolver un problema de alto impacto comercial. Para el equipo desarrollador, representa la consolidación de competencias avanzadas en ingeniería de software, arquitectura de bases de datos relacionales y diseño centrado en el usuario, entregando un producto con un claro retorno de inversión (ROI) para el usuario final.

1.5 Importancia

La importancia de Stock Velia radica en su capacidad de cambiar las reglas del juego comercial para el pequeño empresario. Al blindar el inventario, el software deja de ser un gasto operativo o una herramienta aburrida de control para convertirse en el corazón financiero del negocio. Permite pasar de la reacción tardía (darse cuenta del problema cuando ya no hay dinero) a la prevención inteligente (actuar antes de que el capital se comprometa), otorgando a las PYMES herramientas de grandes corporaciones a un costo accesible y con una interfaz amigable.

1.6 Beneficios

- **Recuperación financiera:** Mitigación de hasta un porcentaje significativo de pérdidas anuales causadas por mermas y errores de conteo.
- **Optimización del flujo de caja:** Transformación del stock estancado en liquidez mediante reportes inteligentes de rotación de productos.
- **Disponibilidad 24/7 y tiempo real:** Actualización automática del inventario de forma simultánea a cada entrada o venta.
- **Tranquilidad operativa:** Eliminación de horas hombre dedicadas a conteos manuales y reducción drástica del margen de error humano.
- **Seguridad y confianza:** Protección estricta de la información comercial mediante sesiones cifradas y control de accesos por roles (RBAC).

1.7 Público objetivo

El sistema está dirigido estratégicamente a dueños, administradores y encargados de inventario de microempresas y PYMES comerciales, tales como:

- Minimercados y tiendas de abastos de barrio.
- Boutiques, almacenes de ropa y calzado.
- Ferreterías y comercios de suministro técnico o repuestos.
- Emprendimientos en fase de expansión que requieran formalizar y blindar su operación logística y financiera.

1.8 Alcance

El alcance del proyecto comprende el diseño, desarrollo, pruebas e implementación de una plataforma web funcional que abarque:

- Módulo de autenticación segura y gestión de usuarios por roles (Administrador / Operador).
- Catálogo maestro de productos con control de categorías y atributos esenciales.
- Motor transaccional para el registro automatizado de entradas y salidas de mercancía en tiempo real.
- Sistema automatizado de alertas visuales ante stock mínimo.
- Módulo analítico de reportes e historial de movimientos, con capacidad de exportación a formatos estándar.
- Despliegue técnico en entorno web bajo protocolos seguros (HTTPS).

1.9 Limitaciones

- **Conectividad a Internet:** Al ser una plataforma basada en web y arquitectura cliente-servidor, el sistema requiere de una conexión estable a internet para garantizar la sincronización en tiempo real de los datos.
- **Gestión de Hardware Externo:** En esta primera fase, el sistema no incluye integración directa nativa con dispositivos de lectura láser avanzados (pistolas de códigos de barra físicas) o impresoras térmicas de recibos de manera automatizada, limitándose a la entrada manual o mediante interfaces web optimizadas.
- **Alcance Financiero Profundo:** El software se enfoca estrictamente en la inteligencia de inventarios y valoración de mercancías, por lo que no pretende sustituir un módulo contable de nómina, impuestos o pasarelas de pago externas complejas.

2. Análisis del Problema

2.1 Situación actual

Las microempresas y PYMES del sector comercial operan en un entorno de alta competitividad donde los márgenes de ganancia son estrechos y el flujo de caja es el motor de supervivencia. Actualmente, la gran mayoría de estos negocios gestionan sus existencias de manera rudimentaria: registros en cuadernos físicos, hojas de cálculo de Excel estáticas y desconectadas, o fundamentadas en la memoria operativa del propietario. Esta desconexión impide tener una visibilidad real y centralizada de los activos, convirtiendo la administración diaria en un proceso lento, reactivo y altamente propenso a errores humanos.

2.2 Problemática encontrada

La raíz de la falla operativa en estos establecimientos radica en la ausencia de un control automatizado, integrado y en tiempo real de los flujos de mercancía. Al no contar con una plataforma centralizada, los administradores operan "a ciegas", descubriendo faltantes, excesos o artículos vencidos únicamente cuando el impacto financiero ya es irreversible, lo que deteriora la rentabilidad del negocio de forma silenciosa.

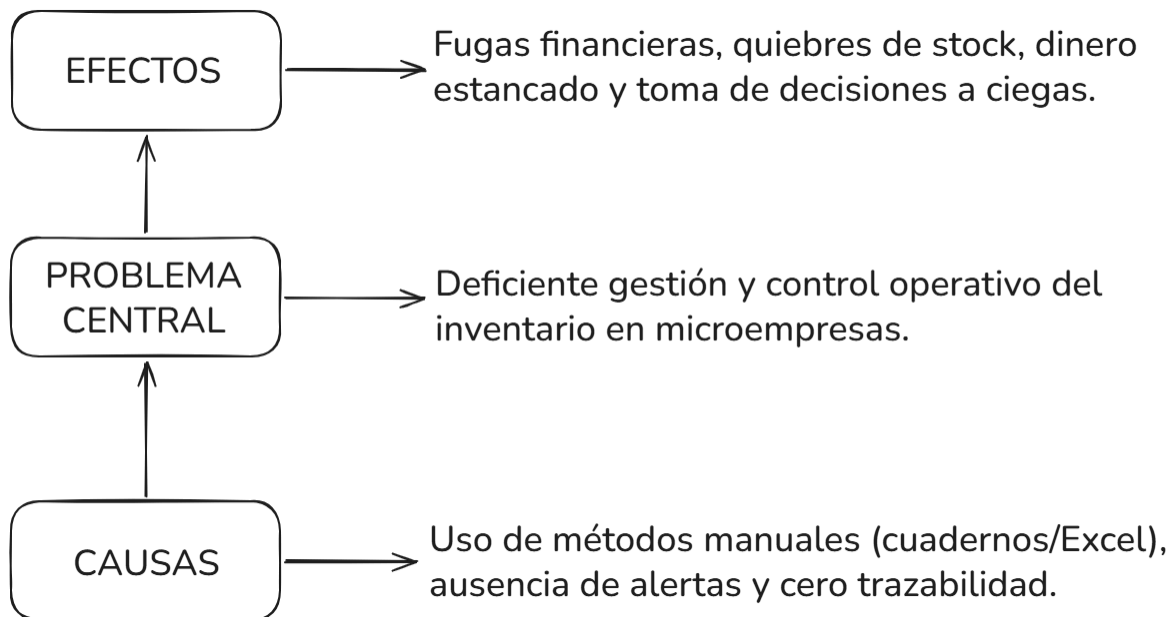
2.3 Causas

- **Empirismo operativo:** Dependencia de métodos manuales y subjetivos para el registro de entradas y salidas de mercancía.
- **Carencia de alertas tempranas:** Inexistencia de mecanismos automatizados que avisen al comerciante sobre los umbrales críticos de stock o productos próximos a caducar.
- **Silos de información:** Desconexión entre el inventario real en estantería y los registros contables o de ventas, generando discrepancias constantes.
- **Falta de trazabilidad:** Ausencia de un historial de auditoría que permita rastrear qué usuario realizó un movimiento o una modificación en el sistema.

2.4 Consecuencias

- **Fugas de capital "hormiga":** Pérdidas de dinero invisibles provocadas por mermas no detectadas, hurtos silenciosos o deterioro de productos.
- **Quiebres de stock (Rotura de inventario):** Incapacidad para satisfacer la demanda del cliente por falta de mercancía en el momento clave, propiciando la fuga de compradores hacia la competencia.
- **Dinero durmiente:** Acumulación de stock estancado que inmoviliza el flujo de caja y bloquea la capacidad de inversión en productos de alta rotación.
- **Incertidumbre gerencial:** Toma de decisiones basada en suposiciones y corazonadas en lugar de métricas financieras y reportes confiables.

2.5 Árbol de problemas



- **Efectos (Impacto Superior):** Pérdida constante de rentabilidad, insatisfacción del cliente por desabastecimiento, estancamiento financiero y riesgo latente de quiebre comercial.
- **Problema Central (Tronco):** Vulnerabilidad y falta de inteligencia en la gestión de inventarios de las microempresas y PYMES.
- **Causas (Raíces):** Procesos manuales propensos a errores, ausencia de automatización en tiempo real y falta de herramientas de auditoría y reportes analíticos.

2.6 Análisis de riesgos

Para garantizar la viabilidad del proyecto, se identificaron los principales riesgos técnicos, operativos y de seguridad, junto con sus respectivas estrategias de mitigación:

Tipo de Riesgo	Descripción del Riesgo	Nivel	Estrategia de Mitigación
Técnico	Caída o interrupción del servicio del servidor backend o base de datos en producción.	Medio	Despliegue en arquitecturas cloud confiables y manejo robusto de excepciones y respaldos automáticos en PostgreSQL.
Seguridad	Vulneración de credenciales o ataques de inyección a las rutas de la API REST.	Alto	Implementación obligatoria de autenticación basada en JWT, hashing de contraseñas con Bcrypt y validaciones estrictas en servidor.

Operativo	Resistencia al cambio por parte de usuarios con bajo nivel de alfabetización digital.	Medio	Diseño de una interfaz de usuario (UI/UX) extremadamente intuitiva, limpia y guiada en React.
Conectividad	Inestabilidad en el acceso a Internet del establecimiento comercial.	Bajo	Optimización del frontend para reducir el peso de carga y manejo eficiente de estados locales transitorios.

2.7 Necesidades identificadas

A partir del estudio del entorno y de los dolores del sector comercial, se determinaron las siguientes necesidades tecnológicas indispensables que Stock Velia entra a solventar:

1. **Monitoreo en tiempo real:** Necesidad imperativa de que cada entrada o venta actualice de forma inmediata y sincronizada el stock global sin retardos.
2. **Seguridad por roles (RBAC):** Restringir el acceso a módulos sensibles (como auditorías o eliminación de registros) únicamente a perfiles autorizados (Administradores vs. Operadores).
3. **Alertas automatizadas:** Un sistema visual preventivo que alerte de forma inmediata al usuario cuando un producto cruce su umbral mínimo de existencia.
4. **Capacidad de auditoría y exportación:** Requerimiento de generar reportes confiables con histórico de movimientos y la facilidad de exportarlos a formatos estándar (PDF o Excel) para respaldos gerenciales.

3. Objetivos

3.1 Objetivo General

Desarrollar y desplegar una plataforma web inteligente de gestión de inventarios y control financiero para microempresas y PYMES, orientada a blindar el capital operativo, automatizar el flujo de mercancías en tiempo real y mitigar las mermas comerciales.

3.2 Objetivo Específicos

- **Diseñar la arquitectura de seguridad y datos:** Implementar un modelo relacional robusto en PostgreSQL junto con autenticación basada en JSON Web Tokens (JWT) y cifrado de credenciales con Bcrypt para garantizar la integridad y confidencialidad del sistema.
- **Construir el núcleo transaccional en tiempo real:** Desarrollar los servicios backend en Node.js con Express y la interfaz responsiva en React para automatizar el registro de entradas y salidas de mercancía de forma concurrente e inmediata.
- **Implementar el motor de alertas y reportes analíticos:** Programar mecanismos de notificación visual ante stock mínimo y módulos de generación y exportación de historiales de inventario a formatos estándar (PDF/Excel).

- **Validar la usabilidad y desempeño del software:** Ejecutar pruebas de calidad (QA), seguridad y usabilidad para asegurar una experiencia óptima en microempresas con bajo nivel de fricción tecnológica.

3.3 Metas

- **Desarrollo del MVP (Producto Mínimo Viable):** Culminar el 100% de los Sprints de programación (backend, frontend y base de datos) dentro del ciclo ágil establecido.
- **Integridad de Datos:** Alcanzar un 0% de pérdida de consistencia transaccional durante las pruebas de entradas y salidas simultáneas en la base de datos.
- **Adopción de Usuario:** Lograr que un usuario sin experiencia técnica previa complete un registro de producto y una transacción de inventario en menos de 2 minutos.
- **Eficiencia Operativa:** Reducir en un 100% el uso de registros manuales en papel o cuadernos dentro de los escenarios de prueba piloto del sistema.

3.4 Indicadores de éxito

- **Tasa de Disponibilidad del Sistema (Uptime):** 99% de operatividad del servidor web y la API REST en entornos de pruebas cloud.
- **Tiempo de Respuesta de la Interfaz:** Menor a 2 segundos en la carga de vistas principales y consultas de stock en React.
- **Precisión de Alertas de Stock:** 100% de efectividad en el disparo automático de notificaciones visuales cuando un producto cruce el umbral mínimo configurado.
- **Satisfacción de Usuario (SUS - System Usability Scale):** Obtener una calificación superior al 85% de favorabilidad en las pruebas de experiencia de usuario con comerciantes simulados.

4. Levantamiento de Requisitos

4.1 Requisitos Funcionales

Los requisitos funcionales describen las acciones, comportamientos y operaciones específicas que el sistema Stock Velia debe ejecutar para satisfacer las necesidades del usuario y procesar la información comercial de manera automatizada.

Código	Módulo / Componente	Nombre del Requisito	Descripción Funcional
RF-01	Seguridad	Registro de Usuarios	El sistema permitirá el alta de nuevos usuarios bajo un rol específico asignado (Administrador u Operador).
RF-02	Seguridad	Autenticación Segura	El sistema validará credenciales de acceso mediante usuario y contraseña cifrada, generando un token de sesión (JWT).

RF-03	Seguridad	Control de Sesión (Logout)	El sistema permitirá cerrar la sesión activa de forma segura, invalidando el token en el cliente y servidor.
RF-04	Seguridad	Gestión de Usuarios (CRUD)	El administrador podrá consultar, modificar datos y realizar borrado lógico de cuentas de usuario.
RF-05	Catálogo	Registro de Productos	El sistema permitirá dar de alta nuevos productos especificando código, categoría, costos, precio de venta y stock mínimo.
RF-06	Catálogo	Modificación de Artículos	El operador o administrador podrá actualizar los atributos de los productos existentes en el inventario.
RF-07	Catálogo	Baja Lógica de Productos	El sistema permitirá discontinuar o realizar borrado lógico de artículos sin perder la trazabilidad histórica.
RF-08	Inventario	Registro de Entradas	El sistema registrará ingresos de mercancía (compras o devoluciones), incrementando automáticamente las existencias.
RF-09	Inventario	Registro de Salidas	El sistema registrará egresos de mercancía (ventas o bajas), descontando existencias de forma concurrente y validando disponibilidad.
RF-10	Inventario	Alertas de Stock Mínimo	El sistema disparará notificaciones visuales automáticas cuando un artículo cruce su umbral mínimo de existencia.
RF-11	Reportes	Historial de Movimientos	El sistema permitirá consultar el registro cronológico de entradas y salidas filtrado por rangos de fecha y producto.
RF-12	Reportes	Exportación de Datos	El sistema ofrecerá la capacidad de exportar reportes consolidados e historiales en formatos estándar como PDF y Excel.

4.2 Requisitos No Funcionales

Los requisitos no funcionales establecen los atributos de calidad, restricciones técnicas, rendimiento, seguridad y usabilidad bajo los cuales debe operar la plataforma.

Código	Categoría	Atributo / Característica	Descripción del Requisito No Funcional
RNF-01	Rendimiento	Tiempo de Respuesta	Las consultas de stock y renderizado de vistas en React deberán completarse en un tiempo inferior a 2 segundos bajo condiciones normales.
RNF-02	Seguridad	Cifrado de Credenciales	Las contraseñas de los usuarios no se almacenarán en texto plano; se procesarán mediante funciones de hash seguro con Bcrypt.
RNF-03	Seguridad	Integridad y Protocolo	Las comunicaciones entre el navegador del cliente y la API REST en Node.js deberán cifrarse mediante el protocolo HTTPS.
RNF-04	Concurrencia	Transaccionalidad	El motor de base de datos (PostgreSQL) garantizará el aislamiento de transacciones simultáneas para evitar incongruencias ante ventas concurrentes.
RNF-05	Usabilidad	Diseño Responsivo (UI/UX)	La interfaz web estará optimizada para adaptarse dinámicamente a diferentes resoluciones de pantalla (escritorio, tablets y móviles).
RNF-06	Mantenibilidad	Arquitectura Modular	El código fuente del backend y frontend se estructurará de manera modular y escalable para facilitar futuras actualizaciones.

4.3 Requisitos de Negocio

Los requisitos de negocio definen las metas de alto nivel, directrices comerciales y normativas operativas que la plataforma Stock Velia debe cumplir para garantizar el éxito y la rentabilidad esperada por el usuario final.

Código	Nombre del Requisito de Negocio	Descripción Comercial y Estratégica
RN-01	Mitigación de Fugas Financieras	La plataforma proveerá auditoría estricta de cada movimiento de inventario para reducir mermas invisibles y diferencias de caja.
RN-02	Accesibilidad Operativa sin Costos Ocultos	El sistema se diseñará ligero y eficiente, minimizando la necesidad de infraestructura costosa para facilitar su adopción en PYMES.
RN-03	Autonomía de Gestión del Comerciante	La interfaz minimizará la curva de aprendizaje a menos de 15 minutos, permitiendo que un comerciante administre su inventario sin fricción.
RN-04	Trazabilidad y Control por Roles (RBAC)	El sistema securizará estrictamente las funciones operativas de las administrativas, garantizando que solo perfiles autorizados ejecuten acciones críticas.

5. Casos de Uso

Los casos de uso modelan el comportamiento funcional del sistema Stock Velia, especificando las interacciones entre los actores del sistema y los procesos automatizados para garantizar el blindaje financiero y la trazabilidad del inventario.

5.1 Actores del Sistema

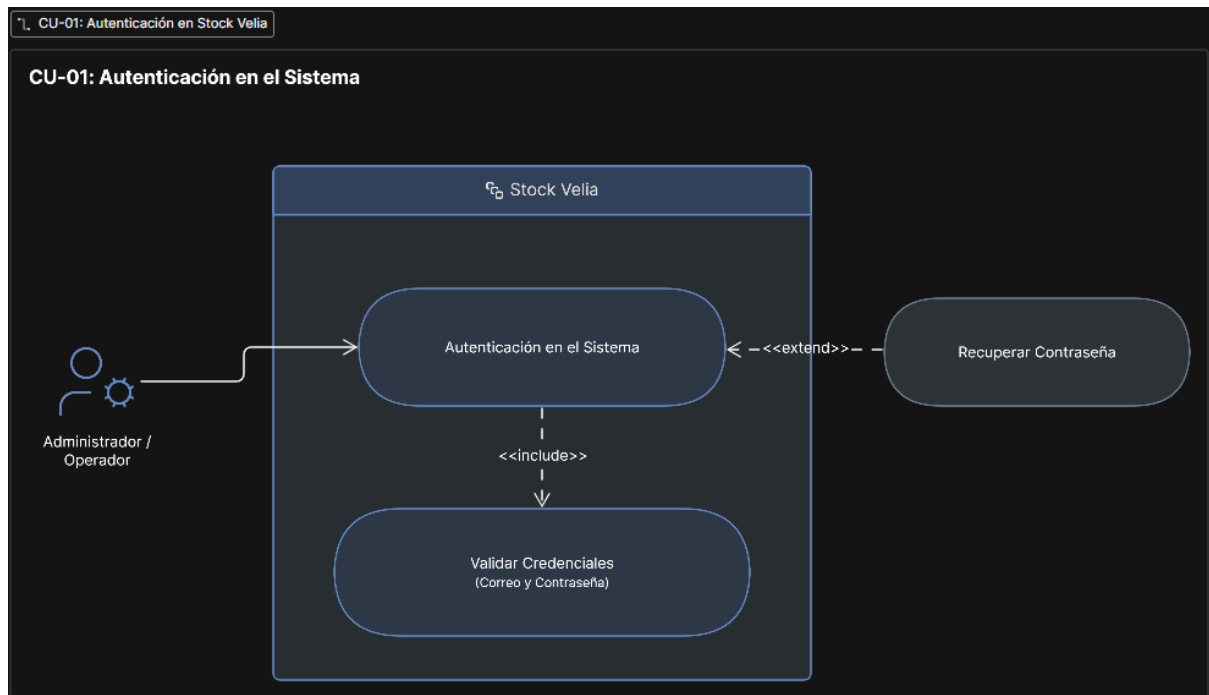
- **Administrador:** Usuario con privilegios gerenciales y absolutos. Gestiona cuentas, configura catálogos globales, revisa auditorías y supervisa el estado financiero del inventario.
- **Operador (Bodeguero / Cajero):** Usuario con privilegios operativos acotados. Ejecuta el registro diario de entradas y salidas de mercancía y consulta existencias en tiempo real.

5.2 Matriz Resumen de Casos de Uso

Código	Nombre del Caso de Uso	Actor Principal	Descripción Breve
CU-01	Autenticación en el Sistema	Administrador / Operador	El usuario ingresa sus credenciales (correo y contraseña) para validar su identidad y acceder al módulo correspondiente según su rol.
CU-02	Gestión de Usuarios (CRUD)	Administrador	El administrador da de alta, edita, consulta o realiza baja lógica de cuentas de usuario en el sistema.
CU-03	Registro de Nuevo Producto	Administrador / Operador	El usuario ingresa un nuevo artículo al catálogo especificando código, costos, precio de venta y stock mínimo.
CU-04	Registro de Entrada de Mercancía	Administrador / Operador	Se registra el ingreso de stock al almacén (por compra a proveedor o devolución), incrementando las existencias globales.
CU-05	Registro de Salida / Venta	Administrador / Operador	Se procesa una salida de inventario (venta o baja), descontando stock de forma concurrente y validando disponibilidad.
CU-06	Generación de Alerta de Stock	Sistema (Automático)	El sistema detecta de forma automática cuando un producto cruza su umbral mínimo y emite una alerta visual en pantalla.
CU-07	Exportación de Reportes	Administrador	El administrador genera y descarga historiales de movimientos e inventario en formatos estándar (PDF o Excel).

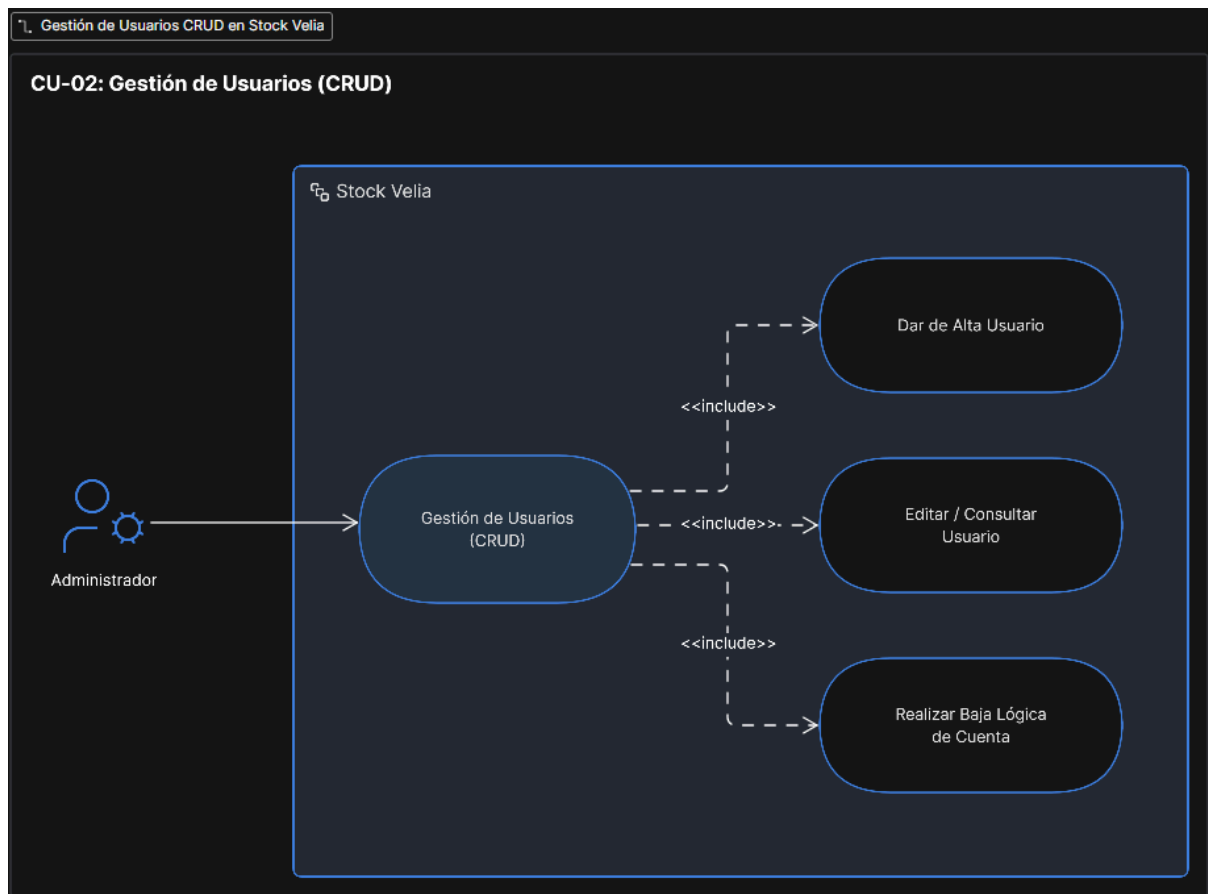
5.3 Especificación Detallada de Casos de Uso Principales

CU-01: Autenticación en el Sistema



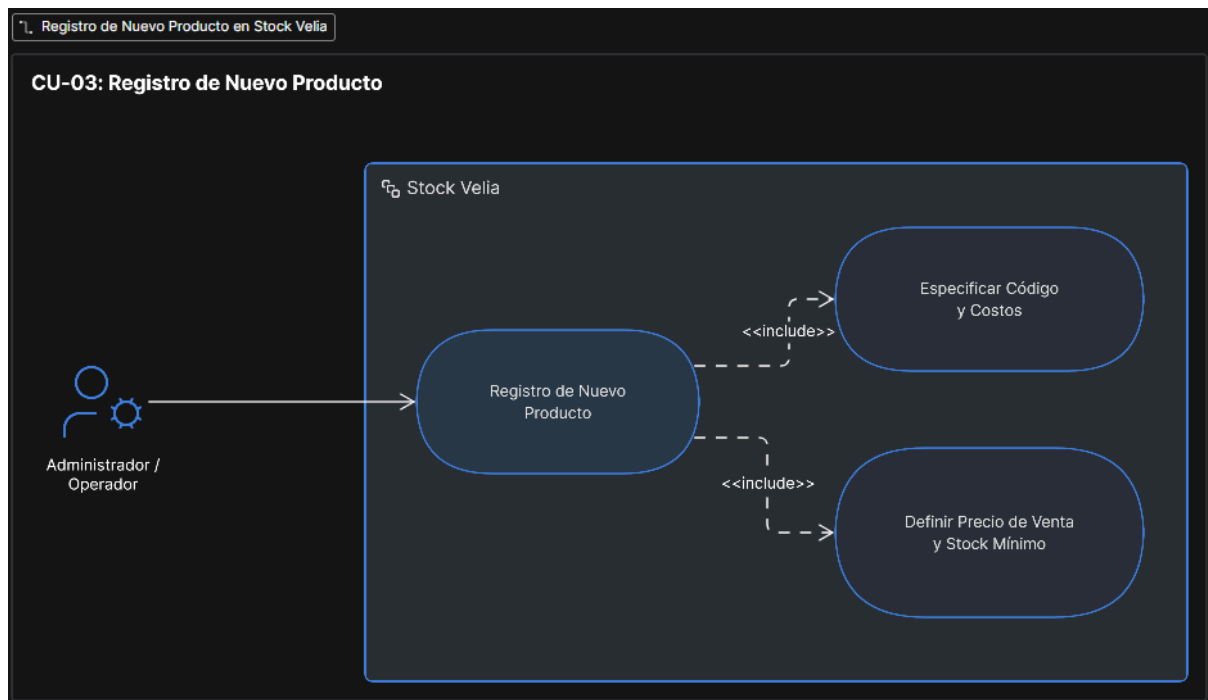
- **Descripción:** Permite al usuario validar sus credenciales de acceso para ingresar a la plataforma con los privilegios correspondientes a su rol.
- **Actor Principal:** Administrador u Operador.
- **Precondiciones:** El usuario debe estar registrado previamente en la base de datos por un administrador.
- **Flujo Principal:**
 - El usuario ingresa su correo electrónico y contraseña en la interfaz de inicio de sesión de React.
 - El sistema envía las credenciales mediante una petición HTTP POST al backend en Node.js.
 - El servidor verifica el hash de la contraseña utilizando Bcrypt y valida los datos en PostgreSQL.
 - El servidor genera un token de sesión JWT y lo devuelve al cliente.
 - El sistema redirige al usuario al panel principal según su rol (Administrador u Operador).
- **Flujos Alternativos (Excepciones):**
 - *Credenciales incorrectas:* Si la contraseña o el correo no coinciden, el sistema deniega el acceso y muestra una alerta visual de error.

CU-02: Gestión de Cuentas de Usuario (CRUD)



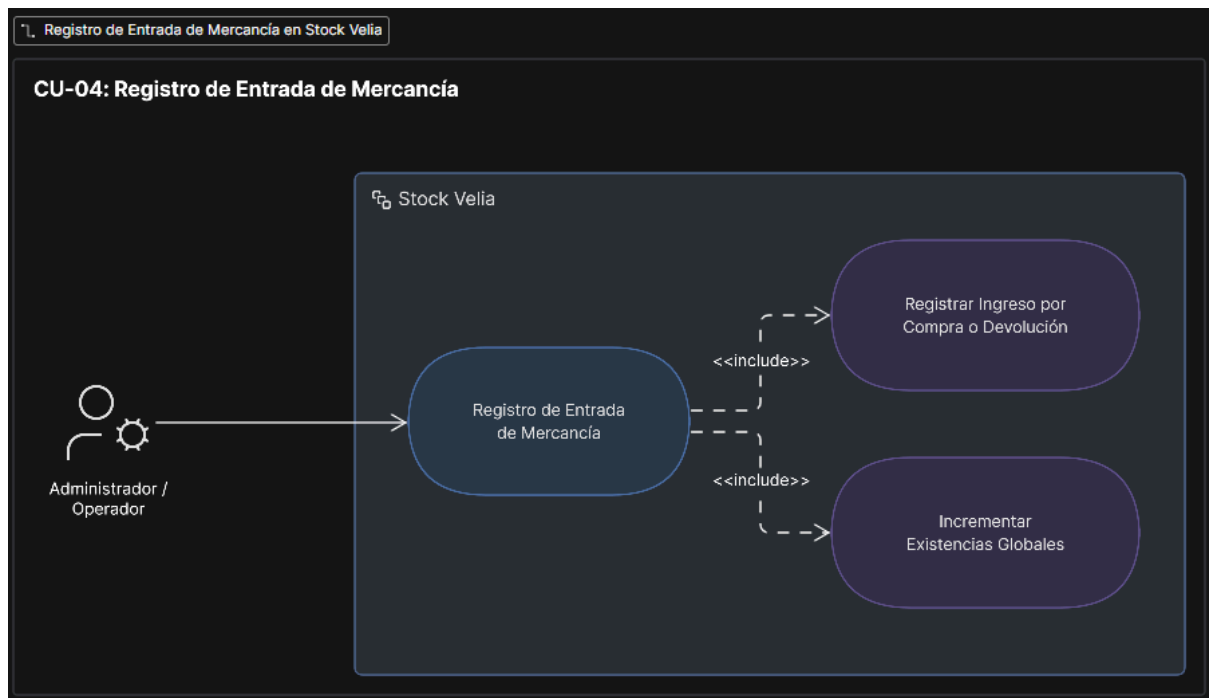
- **Descripción:** Permite al administrador crear, consultar, actualizar o realizar borrado lógico de cuentas de usuarios dentro del sistema.
- **Actor Principal:** Administrador.
- **Precondiciones:** El usuario que ejecuta la acción debe tener el rol de Administrador activo y sesión iniciada.
- **Flujo Principal:**
 1. El administrador accede al módulo de gestión de usuarios en la interfaz.
 2. Selecciona una acción (Crear, Editar, Consultar o Desactivar usuario).
 3. Ingresa o modifica los datos requeridos (nombre, correo, rol asignado).
 4. El backend procesa la solicitud, cifra la contraseña si es un registro nuevo y actualiza el registro en PostgreSQL.
 5. El sistema confirma la operación exitosa y actualiza la lista en pantalla.

CU-03: Registro de Nuevo Producto



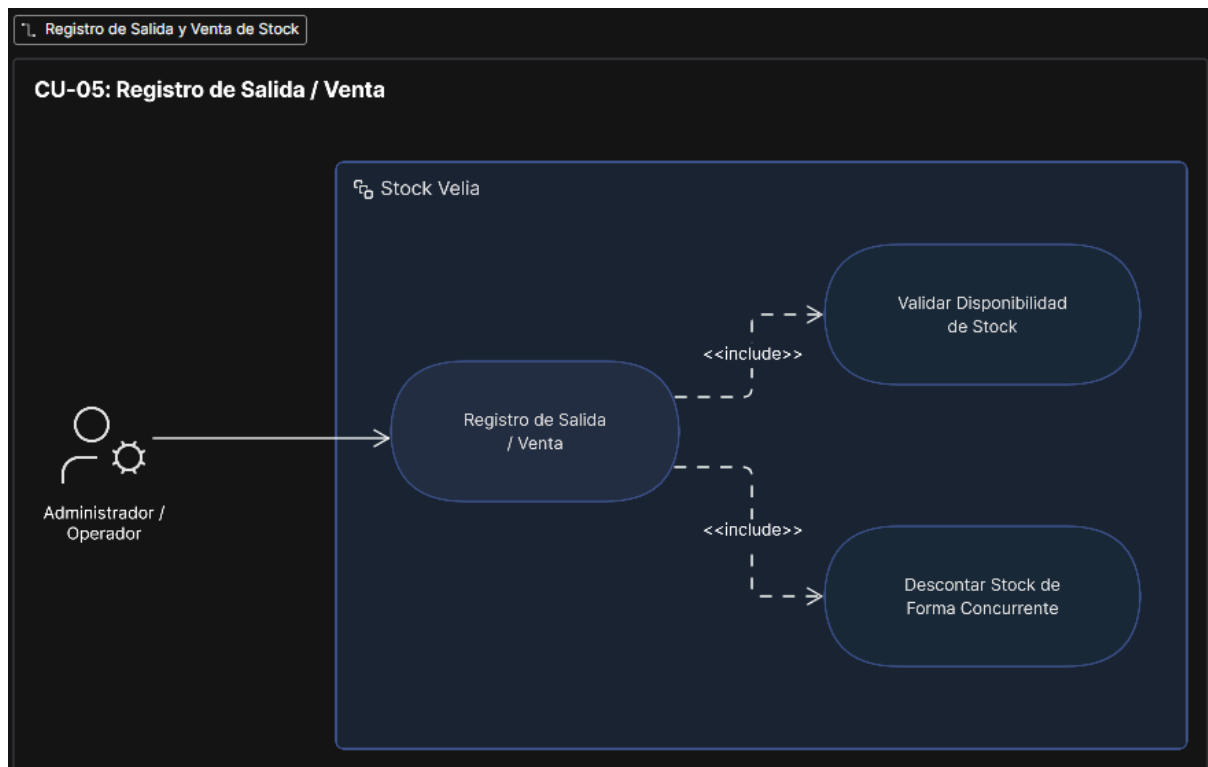
- **Descripción:** Permite dar de alta un artículo comercial en el catálogo global del sistema.
- **Actor Principal:** Administrador u Operador.
- **Precondiciones:** El usuario debe haber iniciado sesión en la plataforma.
- **Flujo Principal:**
 1. El usuario accede al módulo de inventario / catálogo y selecciona "Agregar Producto".
 2. El sistema despliega un formulario solicitando código único, categoría, costos, precio de venta y stock mínimo.
 3. El usuario completa los campos obligatorios y confirma el registro.
 4. El backend valida que el código no esté duplicado y almacena el registro en la base de datos.
 5. El sistema actualiza el catálogo visual y muestra una notificación de éxito.

CU-04: Registro de Entrada de Mercancía



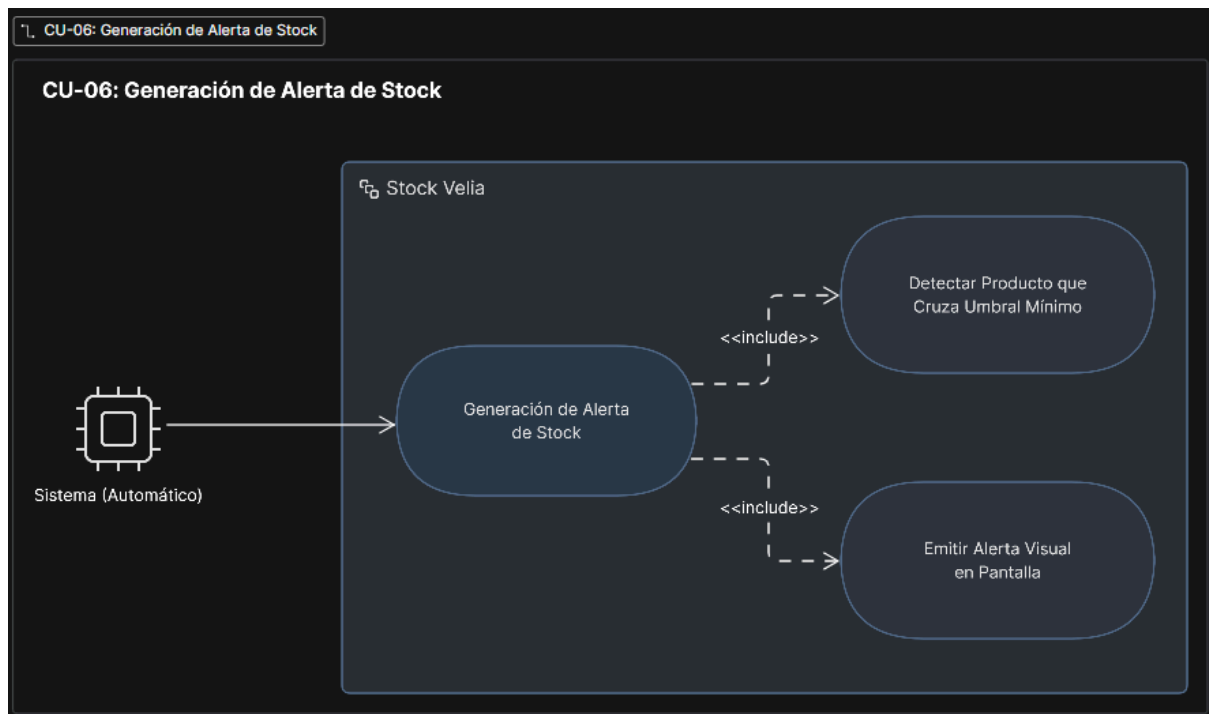
- **Descripción:** Registra el ingreso físico de mercancía al almacén (por compras a proveedores o devoluciones), incrementando las existencias globales.
- **Actor Principal:** Administrador u Operador.
- **Precondiciones:** El producto al que se le sumará stock debe existir previamente en el catálogo.
- **Flujo Principal:**
 1. El usuario accede al módulo de transacciones de entrada.
 2. Busca el producto e ingresa la cantidad exacta de unidades que ingresan.
 3. El sistema valida los datos y ejecuta la transacción en la base de datos sumando el stock actual.
 4. Se registra el movimiento de forma cronológica en la tabla de auditoría.
 5. El sistema muestra la actualización en tiempo real del stock.

CU-05: Registro de Salida / Venta de Mercancía



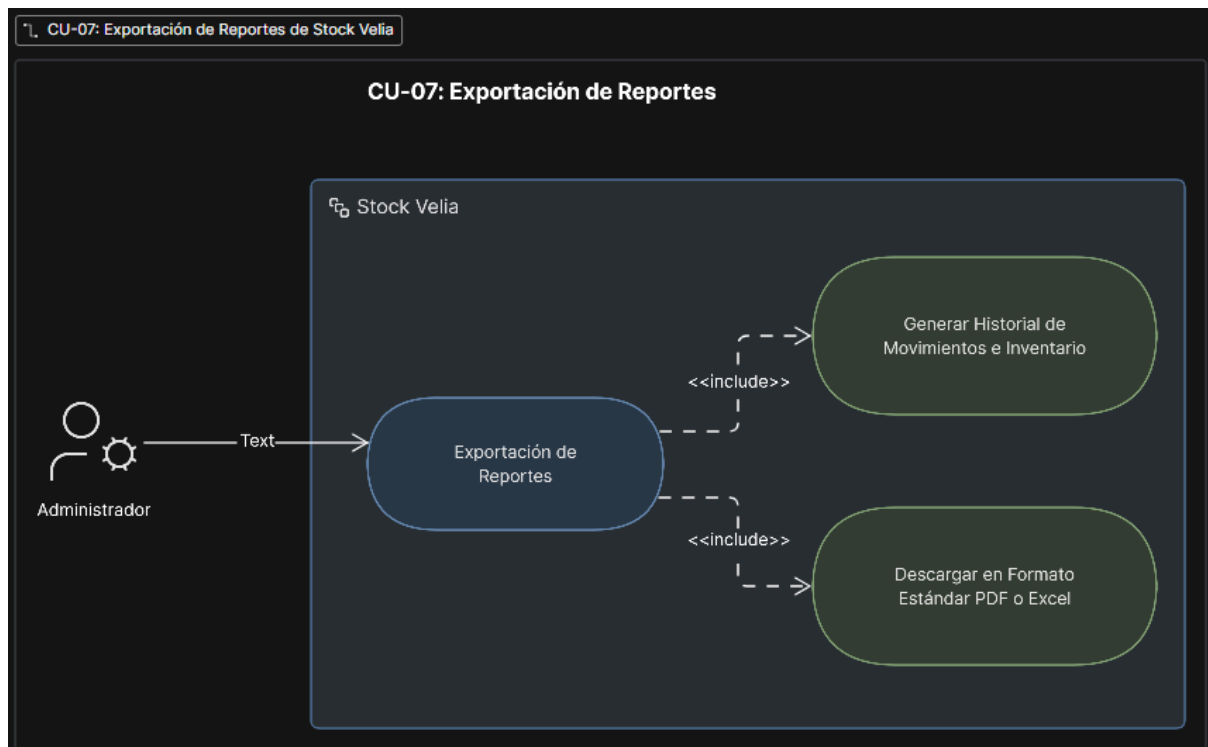
- **Descripción:** Permite registrar la salida de productos del inventario debido a una venta comercial o baja justificada, actualizando el stock de forma inmediata para evitar quiebres financieros.
- **Actor Principal:** Operador o Administrador.
- **Precondiciones:** El usuario debe haber iniciado sesión y el producto debe tener existencias disponibles mayores a cero.
- **Flujo Principal:**
 - El usuario accede al módulo de transacciones/ventas en la interfaz de React.
 - El sistema solicita el código o nombre del producto y la cantidad a egresar.
 - El usuario ingresa los datos y confirma la operación.
 - El backend en Node.js valida mediante PostgreSQL que exista stock suficiente de forma concurrente.
 - El sistema descuenta las existencias, registra el movimiento en el historial de auditoría y muestra un mensaje de éxito.
- **Flujos Alternativos (Excepciones):**
 - *Stock insuficiente:* Si la cantidad solicitada supera las existencias actuales, el sistema rechaza la transacción y emite una advertencia visual al usuario.

CU-06: Generación de Alerta de Stock Mínimo (Automático)



- **Descripción:** Monitorea de manera automática las existencias globales y dispara una notificación visual cuando un producto cruza el umbral crítico definido.
- **Actor Principal:** Sistema (Proceso automatizado).
- **Precondiciones:** El producto debe tener configurado un parámetro de stock mínimo mayor a cero.
- **Flujo Principal:**
 1. Tras cada transacción de salida (CU-05), el backend compara el stock resultante con el límite mínimo configurado.
 2. Si el stock actual es menor o igual al stock mínimo, el sistema marca el producto con un indicador de advertencia.
 3. La interfaz web en React refleja de forma inmediata la alerta visual en el panel del usuario.

CU-07: Exportación de Reportes e Historiales



- **Descripción:** Permite generar y descargar reportes consolidados de los movimientos de inventario en formatos estándar para respaldos gerenciales.
- **Actor Principal:** Administrador.
- **Precondiciones:** El usuario debe poseer privilegios de Administrador y existir registros históricos en el sistema.
- **Flujo Principal:**
 1. El administrador ingresa al módulo de reportes y selecciona los filtros deseados (rango de fechas, tipo de movimiento).
 2. Hace clic en el botón de exportación eligiendo el formato (PDF o Excel).
 3. El backend compila los datos consultados en PostgreSQL y genera el archivo correspondiente.
 4. El navegador descarga el documento automáticamente al equipo del usuario.

Enlaces de Interés:

- [Diagramas - StockVelia](#)
- [Diagramas de Casos de Uso - StockVelia](#)

6. Historias de Usuario

A continuación se presentan detalladamente las Historias de Usuario (HU) del proyecto Stock Velia, estructuradas y organizadas por bloques funcionales tal como se definieron en la documentación técnica del equipo. Cada historia incluye su rol, necesidad, beneficio, criterios de aceptación técnicos y su desglose de tareas.

6.1 Gestión de Seguridad y Usuarios (HU-001 a HU-006)

ID: HU-001 - Registro de Microempresa y Administrador

- **Como:** Emprendedor
- **Quiero:** Registrar mi microempresa y mi perfil de administrador en Stock Velia
- **Para:** Establecer el contexto organizacional y comenzar a gestionar mi inventario digitalmente
- **Puntos de Esfuerzo:** 5
- **Criterios de Aceptación:**
 - Dado que el usuario completa el formulario con nombre de empresa, correo único, contraseña y rol (Administrador), cuando presiona "Registrarse", entonces el sistema valida que el correo no esté duplicado y guarda los datos en la base de datos.
 - Dado que existen campos obligatorios vacíos, cuando se intenta enviar el formulario, entonces el sistema bloquea la acción y muestra mensajes de error específicos.
 - Dado que el registro es exitoso, cuando finaliza el proceso, entonces el sistema

ID: HU-002 - Inicio de Sesión Seguro (JWT)

- **Como:** Usuario Registrado
- **Quiero:** Ingresar mis credenciales de acceso
- **Para:** Entrar al panel de gestión según mis permisos asignados
- **Puntos de Esfuerzo:** 8
- **Criterios de Aceptación:**
 - Dado que el usuario ingresa correo y contraseña válidos, cuando el sistema autentica, entonces genera un token JWT con expiración y crea una sesión activa.
 - Dado que se ingresan credenciales incorrectas, cuando el sistema falla en validar, entonces muestra un mensaje de error genérico para prevenir ingeniería social.
 - Dado que se superan 3 intentos fallidos consecutivos, cuando se intenta el cuarto ingreso, entonces el sistema bloquea temporalmente el acceso por 15 minutos para mitigar ataques de fuerza bruta.

ID: HU-003 - Cierre de Sesión (Logout)

- **Como:** Usuario
- **Quiero:** Cerrar mi sesión voluntariamente
- **Para:** Invalidar mi acceso y proteger la información desde dispositivos compartidos
- **Puntos de Esfuerzo:** 2
- **Criterios de Aceptación:**
 - Dado que el usuario presiona "Cerrar Sesión", cuando la solicitud es procesada, entonces el sistema invalida el token de autenticación en el servidor y limpia el local storage.
 - Dado que la sesión se cierra, cuando se completa la purga de datos, entonces se redirige al usuario a la pantalla de login.

ID: HU-004 - Modificación de Usuarios

- **Como:** Administrador
- **Quiero:** Editar la información de los usuarios existentes (nombre, rol, permisos)
- **Para:** Mantener actualizada la estructura del personal autorizado
- **Puntos de Esfuerzo:** 5
- **Criterios de Aceptación:**

- Dado que el administrador selecciona un perfil de usuario, cuando guarda cambios en rol o estado, entonces el sistema persiste la actualización y genera una entrada en el historial de auditoría indicando quién realizó el cambio.

ID: HU-005 - Eliminación de Usuarios (Borrado Lógico)

- **Como:** Administrador
- **Quiero:** Eliminar usuarios que ya no pertenecen a la empresa
- **Para:** Revocar accesos sin perder la integridad del historial de auditoría
- **Puntos de Esfuerzo:** 3
- **Criterios de Aceptación:**
 - Dado que el administrador solicita eliminar un usuario, cuando confirma la acción, entonces el sistema cambia su estado a "Inactivo" en la base de datos (borrado lógico).
 - Dado que el usuario tiene estado inactivo, cuando intenta loguearse, entonces el sistema deniega el acceso automáticamente.

ID: HU-006 - Control de Acceso por Roles

- **Como:** Sistema
- **Quiero:** Validar los permisos de cada usuario en cada petición a la API
- **Para:** Garantizar que solo personal autorizado ejecute acciones críticas
- **Puntos de Esfuerzo:** 5
- **Criterios de Aceptación:**
 - Dado que un usuario con rol "Empleado" intenta acceder a funciones de "Administrador" (ej. borrar productos), cuando se valida el rol, entonces el sistema bloquea la operación y responde con un error 403 (No autorizado).

ID	Título	Como (Rol)	Quiero	Para	Puntos
HU-001	Registro de Microempresa y Administrador	Emprendedor	Registrar mi microempresa y mi perfil de administrador en Stock Velia	Establecer el contexto organizacional y comenzar a gestionar mi inventario digitalmente	5
HU-002	Inicio de Sesión Seguro (JWT)	Usuario Registrado	Ingresar mis credenciales de acceso	Entrar al panel de gestión según mis permisos asignados	8
HU-003	Cierre de Sesión (Logout)	Usuario	Cerrar mi sesión voluntariamente	Invalidar mi acceso y proteger la información desde dispositivos compartidos	2

HU-004	Modificación de Usuarios	Administrador	Editar la información de los usuarios existentes (nombre, rol, permisos)	Mantener actualizada la estructura del personal autorizado	5
HU-005	Eliminación de Usuarios (Borrado Lógico)	Administrador	Eliminar usuarios que ya no pertenecen a la empresa	Revocar accesos sin perder la integridad del historial de auditoría	3
HU-006	Control de Acceso por Roles	Sistema	Validar los permisos de cada usuario en cada petición a la API	Garantizar que solo personal autorizado ejecute acciones críticas	5

6.2 Gestión de Catálogo de Productos (HU-007 a HU-010)

ID: HU-007 - Registro de Productos con Atributos Completos

- **Como:** Administrador
- **Quiero:** Registrar nuevos productos especificando código, nombre, categoría, precio y stock
- **Para:** Iniciar el control de existencias
- **Puntos de Esfuerzo:** 5
- **Criterios de Aceptación:**
 - Dado que el usuario completa los datos, cuando intenta guardar, entonces el sistema valida que el precio y la cantidad no sean valores negativos.
 - Dado que el código de barras ya existe en la base de datos, cuando se procesa el registro, entonces el sistema muestra un mensaje de error y no duplica el ítem.

ID: HU-008 - Edición de Productos Existentes

- **Como:** Administrador
- **Quiero:** Modificar los atributos de un producto
- **Para:** Mantener el catálogo sincronizado con cambios en precios o descripciones comerciales
- **Puntos de Esfuerzo:** 3
- **Criterios de Aceptación:**
 - Dado que el usuario edita el precio o stock, cuando intenta guardar, entonces el sistema valida que los nuevos valores no sean negativos.
 - Dado que la actualización es exitosa, cuando finaliza, entonces el sistema muestra una notificación de confirmación y actualiza la vista.

ID: HU-009 - Eliminación de Productos (Borrado Lógico)

- **Como:** Administrador
- **Quiero:** Dar de baja productos obsoletos
- **Para:** Mantener organizado el inventario sin comprometer los históricos de movimientos
- **Puntos de Esfuerzo:** 3
- **Criterios de Aceptación:**
 - Dado que el producto tiene movimientos (ventas/entradas) asociados, cuando el administrador intenta eliminarlo, entonces el sistema realiza un borrado lógico (estado: inactivo) para preservar la integridad referencial.
 - Dado que se solicita la eliminación, cuando el proceso concluye, entonces el producto desaparece de la lista activa y se muestra un mensaje de éxito.

ID: HU-010 - Consulta Básica de Productos

- **Como:** Usuario
- **Quiero:** Buscar productos por nombre o filtrar por categoría
- **Para:** Localizar rápidamente información de disponibilidad
- **Puntos de Esfuerzo:** 3
- **Criterios de Aceptación:**
 - Dado que el usuario ingresa un término en la barra de búsqueda, cuando el sistema filtra, entonces muestra coincidencias por nombre, código o categoría en tiempo real.

ID	Título	Cómo (Rol)	Quiero	Para	Puntos
HU-007	Registro de Productos con Atributos Completos	Administrador	Registrar nuevos productos especificando código, nombre, categoría, precio y stock	Iniciar el control de existencias	5
HU-008	Edición de Productos Existentes	Administrador	Modificar los atributos de un producto	Mantener el catálogo sincronizado con cambios en precios o descripciones comerciales	3
HU-009	Eliminación de Productos (Borrado Lógico)	Administrador	Dar de baja productos obsoletos	Mantener organizado el inventario sin comprometer los históricos de movimientos	3
HU-010	Consulta Básica de Productos	Usuario	Buscar productos por nombre o filtrar por categoría	Localizar rápidamente información de disponibilidad	3

6.3 Movimientos e Inventario en Tiempo Real (HU-011 a HU-015)**ID: HU-011 - Registro de Entradas (Compras/Devoluciones)**

- **Como:** Usuario
- **Quiero:** Registrar el ingreso de mercancía
- **Para:** Actualizar el stock disponible automáticamente indicando el motivo y responsable
- **Puntos de Esfuerzo:** 5
- **Criterios de Aceptación:**
 - Dado que se registra una entrada, cuando el usuario completa los campos, entonces el sistema debe capturar obligatoriamente el "Motivo"

(Compra/Devolución), la "Cantidad" y el "Usuario Responsable" (captura automática de sesión).

- Dado que se guarda el movimiento, cuando el proceso termina, entonces el stock del producto aumenta proporcionalmente y se registra en la tabla de movimientos.

ID: HU-012 - Registro de Salidas (Ventas/Bajas)

- **Como:** Usuario
- **Quiero:** Registrar la salida de productos
- **Para:** Descontar automáticamente las existencias tras una venta o daño
- **Puntos de Esfuerzo:** 5
- **Criterios de Aceptación:**
 - Dado que el usuario indica una cantidad de salida, cuando esta supera el stock actual disponible, entonces el sistema bloquea la transacción y muestra una alerta de "Stock Insuficiente".
 - Dado que la salida es por "Baja/Daño", cuando se guarda, entonces se descuenta del stock y se registra el motivo detallado.

ID: HU-013 - Visualización de Stock en Tiempo Real

- **Como:** Usuario
- **Quiero:** Que las cantidades de productos se actualicen automáticamente en mi pantalla
- **Para:** Conocer la disponibilidad inmediata sin recargar el navegador
- **Puntos de Esfuerzo:** 8
- **Criterios de Aceptación:**
 - Dado que ocurre un movimiento de stock en cualquier terminal, cuando el usuario está visualizando el catálogo, entonces el sistema actualiza la cifra visual mediante WebSockets o sondeo corto (polling).

ID: HU-014 - Configuración de Inventario Mínimo

- **Como:** Administrador
- **Quiero:** Definir un umbral de stock mínimo por cada producto
- **Para:** Que el sistema me ayude a prevenir desabastecimientos
- **Puntos de Esfuerzo:** 2
- **Criterios de Aceptación:**
 - Dado que se edita un producto, cuando se establece el campo "Stock Mínimo", entonces el sistema lo guarda como parámetro de referencia para las alertas

ID: HU-015 - Sistema de Alertas por Bajo Stock

- **Como:** Usuario
- **Quiero:** Recibir notificaciones visuales prominentes
- **Para:** Identificar rápidamente los productos que requieren reabastecimiento
- **Puntos de Esfuerzo:** 3
- **Criterios de Aceptación:**

- Dado que el stock actual es menor o igual al stock mínimo, cuando el sistema carga el dashboard o la lista, entonces el ítem se resalta en color rojo o con un icono de advertencia.

ID	Título	Cómo (Rol)	Quiero	Para	Puntos
HU-011	Registro de Entradas (Compras/Devoluciones)	Usuario	Registrar el ingreso de mercancía	Actualizar el stock disponible automáticamente indicando el motivo y responsable	5
HU-012	Registro de Salidas (Ventas/Bajas)	Usuario	Registrar la salida de productos	Descontar automáticamente las existencias tras una venta o daño	5
HU-013	Visualización de Stock en Tiempo Real	Usuario	Que las cantidades de productos se actualicen automáticamente en mi pantalla	Conocer la disponibilidad inmediata sin recargar el navegador	8
HU-014	Configuración de Inventario Mínimo	Administrador	Definir un umbral de stock mínimo por cada producto	Que el sistema me ayude a prevenir desabastecimientos	2
HU-015	Sistema de Alertas por Bajo Stock	Usuario	Recibir notificaciones visuales prominentes	Identificar rápidamente los productos que requieren reabastecimiento	3

6.4 Reportes, Exportación y Búsqueda Avanzada (HU-016 a HU-020)

ID: HU-016 - Generación de Reportes Generales

- **Como:** Administrador
- **Quiero:** Generar un consolidado de existencias
- **Para:** Visualizar el valor total y volumen del inventario actual
- **Puntos de Esfuerzo:** 5
- **Criterios de Aceptación:**
 - Dado que el administrador solicita el reporte, cuando el sistema procesa los datos, entonces muestra totales agregados por categoría y valoración económica (Precio x Stock).

ID: HU-017 - Consulta de Movimientos por Rango de Fechas

- **Como:** Usuario
- **Quiero:** Filtrar el historial de entradas y salidas por fechas
- **Para:** Auditar los movimientos ocurridos en periodos específicos
- **Puntos de Esfuerzo:** 3
- **Criterios de Aceptación:**
 - Dado que se seleccionan fechas de inicio y fin, cuando se aplica el filtro, entonces el sistema muestra solo los registros comprendidos en dicho intervalo.

ID: HU-018 - Historial Individual de Producto

- **Como:** Usuario
- **Quiero:** Ver la trazabilidad de un producto específico
- **Para:** Entender su rotación y cambios sufridos en el tiempo
- **Puntos de Esfuerzo:** 3
- **Criterios de Aceptación:**
 - Dado que se selecciona un ítem, cuando se solicita su detalle, entonces el sistema despliega una línea de tiempo con todas sus transacciones y ediciones registradas.

ID: HU-019 - Exportación de Datos a PDF y Excel

- **Como:** Administrador
- **Quiero:** Descargar los reportes en formatos PDF o Excel
- **Para:** Compartir la información externamente o realizar análisis offline
- **Puntos de Esfuerzo:** 8
- **Criterios de Aceptación:**
 - Dado que se solicita exportación, cuando el sistema procesa la descarga, entonces genera un archivo legible con el formato seleccionado conservando los filtros aplicados.

ID: HU-020 - Búsqueda Avanzada Multi-Criterio

- **Como:** Usuario
- **Quiero:** Combinar múltiples filtros de búsqueda (precio, estado, categoría)
- **Para:** Encontrar productos bajo condiciones complejas
- **Puntos de Esfuerzo:** 5
- **Criterios de Aceptación:**
 - Dado que se aplican varios filtros simultáneos, cuando se ejecuta la búsqueda, entonces el sistema devuelve únicamente los ítems que cumplen con la intersección de todos los criterios.

ID	Título	Como (Rol)	Quiero	Para	Puntos
HU-016	Generación de Reportes Generales	Administrador	Generar un consolidado de existencias	Visualizar el valor total y volumen del inventario actual	5
HU-017	Consulta de Movimientos por Rango de Fechas	Usuario	Filtrar el historial de entradas y salidas por fechas	Auditar los movimientos ocurridos en periodos específicos	3
HU-018	Historial Individual de Producto	Usuario	Ver la trazabilidad de un producto específico	Entender su rotación y cambios sufridos en el tiempo	3
HU-019	Exportación de Datos a PDF y Excel	Administrador	Descargar los reportes en formatos PDF o Excel	Compartir la información externamente o realizar análisis offline	8
HU-020	Búsqueda Avanzada Multi-Criterio	Usuario	Combinar múltiples filtros de búsqueda (precio, estado, categoría)	Encontrar productos bajo condiciones complejas	5

Enlaces de Interés:

1. [Plantillas de Historias de Usuario - StockVelia](#)
2. [Priorización de Historias de Usuario - StockVelia](#)
3. [Cálculo de Horas de Sprints - StockVelia](#)

7. Diagramas UML

7.1 Diagrama de Casos de Uso

El diagrama de casos de uso modela el comportamiento del sistema desde el punto de vista de los usuarios externos (Actores), especificando qué funciones realiza cada rol y cómo interactúan con el núcleo de Stock Velia.

- **Actores del Sistema:**

- **Administrador:** Actor con privilegios elevados. Es el responsable de dar de alta la microempresa, registrar y administrar cuentas de empleados, definir roles, auditar cambios, configurar parámetros globales (como el stock

mínimo) y realizar borrados lógicos de productos o usuarios que ya no operan en el negocio.

- **Empleado (Usuario Operativo):** Actor enfocado en la operatividad diaria. Ingresa al sistema para consultar existencias, registrar los movimientos de entradas y salidas de mercancía, y monitorear las alertas visuales de stock.
- **Casos de Uso Principales y Relaciones (<<include>> / <<extend>>):**
 - **CU-01: Autenticar Usuario:** Permite el acceso seguro mediante credenciales. Como extensión (<<extend>>), incluye el bloqueo temporal tras 3 intentos fallidos y la validación de tokens JWT.
 - **CU-02: Gestionar Catálogo de Productos:** Envuelve las acciones de registrar nuevos ítems con códigos de barras, editar precios y cambiar atributos. Utiliza una relación de inclusión con las validaciones de datos no negativos.
 - **CU-03: Registrar Movimientos de Inventario:** Acceso central donde el empleado especifica si es una entrada (compra/devolución) o salida (venta/daño). Está ligado directamente al cálculo automático del stock.
 - **CU-04: Monitorear Alertas de Inventario:** Caso de uso automatizado donde el sistema evalúa de forma reactiva si el stock actual perfora el umbral mínimo definido, proyectando avisos visuales en tiempo real.
 - **CU-05: Generar y Exportar Reportes:** Permite al administrador consolidar existencias, filtrar por rangos de fecha y exportar la información estructurada a documentos PDF u hojas de cálculo Excel.

Enlaces de Interés:

- [Diagrama de Casos de Uso - StockVelia](#)

7.2 Diagrama de Clases

Este diagrama estructural define el dominio de la aplicación mediante las entidades persistentes, sus atributos tipados, métodos y las relaciones de cardinalidad que rigen la base de datos relacional de Stock Velia.

1. Clase Empresa

- **Atributos Principales:** id_empresa (PK), nombre, correo, fecha_registro, estado.
- **Relaciones:** Se vincula uno a muchos (1:N) con Usuario y con Producto.

2. Clase Rol y Permiso

- **Atributos Principales:**
 - **Rol:** id_rol (PK), nombre.
 - **Permiso:** id_permiso (PK), nombre, descripcion.
- **Relaciones:** Se relacionan mediante la tabla intermedia RolPermiso (N:M) y definen los privilegios del sistema.

3. Clase Usuario

- **Atributos Principales:** id_usuario (PK), id_empresa (FK), id_rol (FK), nombre, correo, password, intentos_fallidos, bloqueo_hasta, estado.
- **Relaciones:** Pertenece a una Empresa y a un Rol. Ejecuta MovimientosInventario y genera registros en Auditoria.

4. Clase Categoría y Producto

- **Atributos Principales:**

- **Categoría:** id_categoria (PK), nombre, descripcion.
- **Producto:** id_producto (PK), id_empresa (FK), id_categoria (FK), codigo, nombre, precio, stock, stock_minimo, estado.

- **Relaciones:** Una Categoría agrupa múltiples Productos. Cada producto pertenece a una empresa y acumula un historial de movimientos.

5. Clase MovimientoInventario

- **Atributos Principales:** id_movimiento (PK), id_producto (FK), id_usuario (FK), tipo_movimiento (ENTRADA/SALIDA/BAJA), motivo, cantidad, observacion.
- **Relaciones:** Relaciona al Producto afectado con el Usuario responsable de la operación. Activa el trigger de actualización automática de stock.

6. Tablas de Soporte y Seguridad (Auditoria, TokenBlacklist,CodigoRecuperacion)

- **Responsabilidad:** Gestionan de manera transversal la trazabilidad de acciones administrativas de los usuarios (Auditoria), la invalidación de sesiones al hacer logout (TokenBlacklist), y el almacenamiento temporal de códigos de verificación por correo (CodigoRecuperacion).

Enlaces de Interés:

- [Diagrama de Clases - StockVelia](#)

7.3 Diagrama de Secuencia

El diagrama de secuencia detalla la interacción basada en el tiempo y el intercambio de mensajes entre los componentes de software ante un escenario crítico del negocio: el Registro de una Salida de Inventario (Venta).

1. **Solicitud Inicial:** El usuario interactúa con la interfaz gráfica de React (Frontend), selecciona un producto, introduce la cantidad que va a salir y presiona el botón "Registrar Salida".
2. **Petición HTTP:** El Frontend emite una solicitud asíncrona HTTP POST (/api/movimientos) adjuntando el token JWT de la sesión activa en las cabeceras de autorización.
3. **Validación de Seguridad:** El **Middleware de Autenticación** intercepta la petición, verifica la firma del JWT y comprueba que el usuario posea los permisos necesarios para modificar inventarios.
4. **Verificación de Negocio:** El **Controlador de Movimientos (Backend)** ejecuta una consulta a la base de datos para comprobar la disponibilidad: evalúa si stock_actual >= cantidad_solicitada.
 - **Ruta Alternativa (Error):** Si el stock es insuficiente, el servidor retorna inmediatamente un código de respuesta HTTP 400 Bad Request con el mensaje descriptivo "Stock Insuficiente", deteniendo el flujo y mostrando una alerta visual en el navegador.

5. **Transacción Atómica en Base de Datos:** Si hay existencias suficientes, el servidor abre una transacción SQL para garantizar atomicidad:
 - Descuenta la cantidad en la tabla Producto.
 - Inserta un nuevo registro en la tabla Movimiento_Inventario.
6. **Notificación en Tiempo Real:** Mediante el servidor de **WebSockets (Socket.io)**, el backend emite un evento global a todas las terminales conectadas para que el catálogo refleje el nuevo stock de manera instantánea sin necesidad de recargar la página.
7. **Respuesta Exitosa:** El backend responde al cliente original con un código HTTP 201 Created, permitiendo que el Frontend limpie el formulario y despliegue un aviso de éxito al operador.

Enlaces de Interés:

- [Diagrama de Secuencia - StockVelia](#)

7.4 Diagrama de Actividades

Este diagrama de comportamiento modela paso a paso el flujo lógico y las ramificaciones condicionales que ocurren en el sistema durante el **Control y Alerta de Stock Mínimo**.

1. **Punto de Partida:** Se efectúa y almacena con éxito un nuevo movimiento de inventario (bien sea una entrada o una salida).
2. **Proceso de Evaluación:** El motor lógico del backend recupera el valor actualizado del campo stock_actual y lo contrasta directamente con el parámetro stock_minimo configurado para dicho producto.
3. **Punto de Decisión (Rombo de Control):**
 - **Condición A (stock_actual <= stock_minimo):** El flujo toma la ruta de advertencia. El sistema activa una marca lógica de criticidad, almacena una notificación interna y ordena a la interfaz de usuario resaltar el ítem en color rojo junto a un icono de advertencia prominente.
 - **Condición B (stock_actual > stock_minimo):** El flujo toma la ruta estándar. El sistema mantiene o restablece el estado visual normal del producto en el panel de control.
4. **Punto de Convergencia:** El proceso de control finaliza y la interfaz queda actualizada y sincronizada para el usuario.

Enlaces de Interés:

- [Diagrama de Actividades - StockVelia](#)

7.5 Diagrama de Estados

Representa la máquina de estados por la que transita un **Producto** a lo largo de su ciclo de vida operativo dentro de la plataforma Stock Velia.

- **Estado [Registrado / Creado]:** El administrador da de alta el producto ingresando sus datos iniciales. El stock se define en cero o en su cantidad inicial y el producto queda habilitado.
- **Estado [Activo / En Stock]:** Es el estado operativo normal. El producto cuenta con un nivel de existencias superior al mínimo establecido ($\text{stock_actual} > \text{stock_minimo}$), permitiendo realizar transacciones de entrada y salida sin restricciones.
- **Estado [Crítico / Stock Bajo]:** Se activa de forma automática cuando una venta o salida reduce las existencias al umbral del stock mínimo o por debajo de este ($\text{stock_actual} \leq \text{stock_minimo}$). El sistema exhibe alertas visuales permanentes para urgir el reabastecimiento.
- **Estado [Agotado]:** Se alcanza cuando el inventario llega estrictamente a cero ($\text{stock_actual} == 0$). En este estado, el sistema bloquea cualquier intento de registrar nuevas salidas para prevenir números negativos, permitiendo únicamente transacciones de entrada (compras o devoluciones).
- **Estado [Inactivo / Borrado Lógico]:** Ocurre cuando el administrador decide dar de baja el producto por obsolescencia comercial. El registro no se borra físicamente de la base de datos para no corromper los reportes históricos de movimientos pasados, pero se oculta de los listados activos de ventas.

Enlaces de Interés:

- [Diagrama de Estados - StockVelia](#)

7.6 Diagrama de Componentes

Muestra la organización modular, la arquitectura de capas y las dependencias físicas/lógicas entre los bloques de software que componen la solución tecnológica.

- **Capa de Presentación (Frontend SPA):**
 - Construida con React y Vite. Contiene los componentes visuales de interfaz organizados por vistas funcionales (Módulo de Autenticación, Panel de Control, Módulo de Catálogo, Módulo de Movimientos y Módulo de Reportes).
 - Incluye gestores de estado reactivo y clientes de comunicación HTTP/WebSocket.
- **Capa de Lógica de Negocio y Servidor (Backend API):**
 - Desarrollada en **Node.js con Express**.
 - **Subcomponentes:** Módulo de Rutas (Endpoints REST), Controladores de Negocio, Middlewares de Seguridad (Validación de JWT y RBAC de roles), y el Servidor de Eventos en Tiempo Real (**Socket.io**).

- **Subsistema de Reportes:** Librerías dedicadas a la renderización y exportación de archivos en formatos **jsPDF** y **ExcelJS**.
- **Capa de Persistencia de Datos:**
 - Gestor de Base de Datos Relacional (**PostgreSQL**), encargada de mantener la integridad referencial mediante claves foráneas, restricciones de unicidad y transacciones ACID.

Enlaces de Interés:

- [Diagrama de Componentes - StockVelia](#)

7.7 Diagrama de Despliegue

Describe la topología de la infraestructura física y virtual, mapeando los nodos de hardware y software donde se despliega y ejecuta la arquitectura de StockVelia en un entorno de producción en la nube.

- **Nodo Cliente (Dispositivo del Usuario Final):**
 - Representa el ordenador, portátil o tableta del emprendedor o empleado.
 - Ejecuta un Navegador Web Moderno que procesa la aplicación cliente (SPA) y establece conexiones cifradas mediante los protocolos HTTPS (para peticiones de la API) y WSS (para la sincronización bidireccional de stock mediante WebSockets).
- **Nodo Servidor en la Nube (Plataforma de Hosting - Ej. Render / Railway / AWS):**
 - **Contenedor de Aplicación Frontend:** Aloja los archivos estáticos optimizados (HTML, CSS, JS) servidos a través de un proxy inverso ligero (Nginx).
 - **Contenedor de Servidor Backend:** Aloja el entorno de ejecución de **Node.js**, corriendo simultáneamente el servidor HTTP de la API REST y el motor de WebSockets.
- **Nodo de Base de Datos Gestionada (Cloud Database - Ej. Supabase / AWS RDS PostgreSQL):**
 - Servidor dedicado y aislado que almacena de forma segura la información relacional de las microempresas, catálogos, usuarios e historiales de movimientos, garantizando copias de seguridad automáticas y alta disponibilidad.

Enlaces de Interés:

- [Diagrama de Despliegue - StockVelia](#)

7.8 Diagrama de Paquetes

Organiza lógicamente el código fuente del proyecto backend y frontend en paquetes o directorios cohesivos, facilitando la mantenibilidad, el principio de responsabilidad única y la escalabilidad del desarrollo.

- **Paquete config:** Agrupa los archivos de configuración inicial, variables de entorno (.env) y los parámetros de conexión con el gestor de base de datos.
- **Paquete controllers:** Contiene la lógica central que procesa las peticiones HTTP recibidas desde las rutas, orquestando las reglas de negocio antes de interactuar con la base de datos.
- **Paquete models:** Define las estructuras de datos, esquemas relacionales y consultas SQL/ORM para interactuar directamente con las tablas de la base de datos (Empresa, Usuario, Producto, Movimiento).
- **Paquete routes:** Define los puntos de entrada (endpoints) de la API RESTful, enrutando cada petición HTTP al controlador correspondiente según el verbo y la URL solicitada.
- **Paquete middlewares:** Aloja funciones intermedias de uso transversal, tales como el filtro de verificación de tokens JWT, los interceptores de control de acceso por roles (RBAC) y los validadores de esquemas de entrada de datos.
- **Paquete utils o helpers:** Contiene funciones de apoyo reutilizables, incluyendo los generadores de hashes criptográficos para contraseñas (Bcrypt) y los motores de estructuración de archivos para la exportación de reportes en PDF y Excel.

Enlaces de Interés:

- [Diagrama de Paquetes - StockVelia](#)

8. Arquitectura del Sistema

8.1 Arquitectura General

La arquitectura general de **Stock Velia** se rige bajo el patrón de diseño **Modelo-Vista-Controlador (MVC)**, estructurado en un entorno web moderno de cliente-servidor con comunicación síncrona (REST) y asíncrona en tiempo real (WebSockets).

- **Esquema de Interacción MVC:**
 - **La Vista (View):** Representada por la aplicación de interfaz de usuario en el cliente (**React + Vite**), encargada de renderizar los componentes visuales, formularios y paneles de control en tiempo real.
 - **El Controlador (Controller):** Ubicado en el servidor backend (**Node.js con Express**), intercepta las peticiones del usuario, procesa la lógica de negocio, valida los datos y coordina la comunicación con la base de datos.
 - **El Modelo (Model):** Representado por las entidades relacionales de la base de datos (**PostgreSQL**), los esquemas y las reglas de integridad referencial (incluyendo restricciones, triggers y validaciones lógicas).

8.2 Arquitectura por Capas

El software está desglosado organizadamente para separar las responsabilidades bajo el paradigma MVC y garantizar un código mantenible:

- **1. Capa de la Vista (Frontend - React):**
 - Responsable de la presentación visual y la interacción directa con el usuario (empleado o administrador).
 - Renderiza dinámicamente el catálogo, los movimientos y las alertas, y actualiza la pantalla de forma reactiva gracias al cliente de WebSockets.
- **2. Capa del Controlador (Backend - Controladores Express):**
 - Módulos lógicos encargados de recibir las peticiones HTTP (p. ej., control de usuarios, productos y movimientos).
 - Orquestan el flujo de las operaciones, invocan las reglas de negocio y responden al cliente con los códigos HTTP adecuados.
- **3. Capa del Modelo y Persistencia (Base de Datos - PostgreSQL):**
 - Define la estructura de los datos (tablas como productos, usuario, movimientos_inventario, etc.).
 - Ejecuta la lógica transaccional mediante restricciones ACID y el trigger PL/pgSQL encargado de actualizar automáticamente el stock y validar existencias negativas.
- **4. Capa Transversal de Seguridad y Middlewares:**
 - Componentes que actúan antes de llegar a los controladores, validando los tokens **JWT**, verificando los permisos de acceso por roles (**RBAC**) y sanitizando las entradas.

8.3 Arquitectura Elegida

Se ha seleccionado un Modelo Monolítico Modular basado en el Patrón MVC con API REST y WebSockets, ideal por las siguientes razones:

- **Cohesión con el Dominio:** El patrón MVC proporciona una separación de poderes sumamente natural y limpia para aplicaciones de gestión empresarial como Stock Velia, donde las vistas de inventario dependen directamente de los modelos de datos transaccionales.
- **Simplicidad y Eficiencia Operativa:** Al estar orientado a microempresas, mantener un monolito con arquitectura MVC evita la sobre ingeniería de microservicios, facilitando enormemente el despliegue, las pruebas locales y el mantenimiento evolutivo.

- **Sincronización en Tiempo Real:** La combinación del flujo MVC clásico con eventos de WebSockets (Socket.io) permite que cualquier cambio realizado en el Controlador (como una venta o entrada de mercancía) se propague inmediatamente a las Vistas de todos los usuarios conectados sin necesidad de recargar la página.

9. Diseño de Bases de Datos

9.1 Modelo Conceptual

El modelo conceptual representa la abstracción de alto nivel de los datos del negocio, identificando las entidades principales del sistema, sus características generales y cómo interactúan entre sí sin entrar en detalles de motores de bases de datos específicos:

- **Entidades Principales:**
 - **Empresa:** Entidad organizativa central que agrupa a los usuarios, productos y operaciones.
 - **Usuario:** Persona autorizada que interactúa con el sistema bajo un rol y permisos definidos.
 - **Rol / Permiso:** Conjunto de privilegios que determinan el control de acceso.
 - **Categoría:** Clasificación o familia a la que pertenecen los productos del inventario.
 - **Producto:** Bien o mercancía física controlada dentro del stock de la empresa.
 - **Movimiento de Inventario:** Registro histórico de entradas, salidas o bajas de mercancía.
- **Relaciones Conceptuales:** Una *Empresa* posee múltiples *Usuarios* y *Productos*. Cada *Usuario* tiene un *Rol*. Los *Productos* pertenecen a una *Categoría* y generan múltiples *Movimientos de Inventario* ejecutados por los *Usuarios*.

9.2 Modelo Lógico

El modelo lógico detalla la estructura relacional de los datos, especificando las tablas, columnas, tipos de datos abstractos y las reglas de normalización (hasta la Tercera Forma Normal - 3NF) para evitar redundancias:

- **Tablas Nucleares:** empresa, roles, permisos, roles_permisos (relación N:M), categorías, usuario.
- **Tablas Operacionales:** productos, movimientos_inventario.
- **Tablas de Soporte y Seguridad:** auditoria, tokens_blacklist, codigos_recuperacion.
- **Integridad:** Se aplican llaves primarias (PK) en cada tabla para garantizar la unicidad de los registros y llaves foráneas (FK) para mantener la integridad referencial cruzada (por ejemplo, relacionando usuario con empresa y roles).

9.3 Modelo Físico

El modelo físico traduce el diseño lógico a la sintaxis y especificaciones concretas del motor relacional **PostgreSQL**, incorporando optimizaciones de rendimiento y restricciones de almacenamiento:

- **Estrategia de Identificadores:** Uso de identificadores únicos en formato texto formateado (VARCHAR(20)) generados mediante una función PL/pgSQL y una secuencia global (generar_id_formateado()).
- **Tipos de Datos y Restricciones Físicas:** Empleo de tipos nativos (VARCHAR, INTEGER, NUMERIC, TIMESTAMP, BOOLEAN, TEXT).
- **Automatización Transaccional (Triggers):** Incorporación del disparador trg_actualizar_stock que ejecuta la función actualizar_stock(), encargada de validar que no existan existencias negativas y recalcular automáticamente el stock en la tabla productos cada vez que se inserta un movimiento_inventario.

9.4 Diagrama Entidad Relación

Representación esquemática de la interconexión entre las tablas del sistema:

- **empresa (1) —< (N) usuario** (Una empresa tiene muchos usuarios).
- **roles (1) —< (N) usuario** (Un rol es asignado a muchos usuarios).
- **roles (N) >—< (N) permisos** (A través de la tabla intermedia roles_permisos).
- **empresa (1) —< (N) productos** (Una empresa administra un catálogo de productos).
- **categorias (1) —< (N) productos** (Una categoría agrupa múltiples productos).
- **productos (1) —< (N) movimientos_inventario** (Un producto registra un historial de múltiples entradas o salidas).
- **usuario (1) —< (N) movimientos_inventario** (Un usuario es responsable de registrar los movimientos).
- **usuario (1) —< (N) auditoria** (El sistema audita las acciones ejecutadas por los usuarios).

Enlaces de Interés:

- [Diagrama Modelo Entidad Relación - StockVelia](#)

9.5 Diccionario de Datos:

A continuación se documentan las tablas más críticas del sistema bajo la estructura solicitada (tomando como referencia principal las tablas de negocio productos y movimientos_inventario):

A. Tabla: productos

- **Nombre:** productos
- **Descripción:** Almacena el catálogo de mercancías y existencias activas e inactivas de la microempresa.
- **Campos:** id_producto, id_empresa, id_categoria, codigo, nombre, descripcion, precio, stock, stock_minimo, estado, fecha_creacion.

- **Tipo de dato:**
 - **id_producto:** VARCHAR(20)
 - **id_empresa:** VARCHAR(20)
 - **id_categoria:** VARCHAR(20)
 - **codigo:** VARCHAR(50)
 - **nombre:** VARCHAR(150)
 - **descripcion:** TEXT
 - **precio:** NUMERIC(12, 2)
 - **stock:** INTEGER
 - **stock_minimo:** INTEGER
 - **estado:** BOOLEAN
 - **fecha_creacion:** TIMESTAMP
- **Restricciones:** NOT NULL en campos clave, UNIQUE en codigo, y validaciones CHECK

```
(precio >= 0), CHECK(stock >= 0), CHECK(stock_minimo >= 0).
```
- **Relaciones:** Pertenece a una empresa (N:1) y a una categoria (N:1); es padre de movimientos_inventario (1:N).
- **Índices:** Índice implícito por la llave primaria y clave única en codigo.
- **Claves primarias:** id_producto
- **Claves foráneas:** id_empresa (referencia a empresa), id_categoria (referencia a categorias).

B. Tabla: movimientos_inventario

- **Nombre:** movimientos_inventario
- **Descripción:** Registra de forma inalterable cada transacción de entrada, salida o baja de mercancía que afecta el stock.
- **Campos:** id_movimiento, id_producto, id_usuario, tipo_movimiento, motivo, cantidad, observacion.
- **Tipo de dato:**
 - **id_movimiento:** VARCHAR(20)
 - **id_producto:** VARCHAR(20)
 - **id_usuario:** VARCHAR(20)
 - **tipo_movimiento:** VARCHAR(20)
 - **motivo:** VARCHAR(50)
 - **cantidad:** INTEGER
 - **observacion:** TEXT
- **Restricciones:** NOT NULL general, validación CHECK

```
(tipo_movimiento IN ('ENTRADA', 'SALIDA', 'BAJA'))
```

, y CHECK

```
(cantidad > 0)
```

.
- **Relaciones:** Depende de un producto (N:1) y de un usuario responsable (N:1).
- **Índices:** Índice por llave primaria y optimización para búsquedas por producto.
- **Claves primarias:** id_movimiento
- **Claves foráneas:** id_producto (referencia a productos), id_usuario (referencia a usuario).

Enlaces de Interés:

- [Script de la Base de Datos - StockVelia](#)

10. Normalización de la Base de Datos

El diseño relacional de la base de datos de Stock Velia ha sido optimizado aplicando las Tres Primeras Formas Normales (3NF). Esto permite evitar anomalías en las operaciones de inserción, actualización y borrado, garantizando un almacenamiento eficiente y seguro de los datos de inventario y usuarios.

10.1 Primera Forma Normal (1NF)

Una tabla se encuentra en 1NF si todos sus atributos contienen valores atómicos (indivisibles) y no existen grupos repetitivos o matrices dentro de los campos.

- **Aplicación en Stock Velia:**
 - En tablas como productos, atributos como el precio, el stock y el stock mínimo almacenan un único valor numérico escalar.
 - En la tabla movimientos_inventario, campos como la cantidad, el motivo y el tipo de movimiento están completamente desglosados sin listas ni arreglos anidados.
 - Todas las filas de las tablas poseen una clave primaria única (id_producto, id_usuario, etc.) que identifica de forma inequívoca cada registro.

10.2 Segunda Forma Normal (2NF)

Una tabla se encuentra en 2NF si está en 1NF y todos sus atributos no clave dependen funcionalmente por completo de la clave primaria (se eliminan las dependencias parciales, especialmente relevante en tablas con claves compuestas).

- **Aplicación en Stock Velia:**
 - En la tabla intermedia roles_permisos, cuya clave primaria es compuesta (id_rol, id_permiso), no existen atributos descriptivos adicionales que dependan parcialmente de solo uno de los dos identificadores; la tabla existe exclusivamente para relacionar ambos conceptos de manera pura.
 - En las tablas con clave primaria simple (como productos o usuario), todos los atributos descriptivos (nombre, correo, precio, stock) dependen directamente y en su totalidad de la clave primaria de la entidad, evitando redundancias asociadas a llaves candidatas parciales.

10.3 Tercera Forma Normal (3NF)

Una tabla se encuentra en 3NF si está en 2NF y no existen dependencias transitivas entre los atributos que no son clave (es decir, ningún atributo no clave depende de otro atributo no clave).

- **Aplicación en Stock Velia:**

- Para evitar la duplicación de datos organizacionales o de clasificación, la información no se almacena de forma plana en una sola tabla de productos. En su lugar, se separaron las entidades empresa y categorías.
- La tabla productos almacena únicamente las llaves foráneas id_empresa e id_categoria. Si el nombre o la descripción de una categoría cambian, solo se modifica en la tabla categorías y no producto por producto, eliminando las anomalías de actualización.
- De igual forma, los permisos del sistema están desacoplados en la tabla permisos y se vinculan a los roles mediante roles_permisos, evitando dependencias transitivas directas sobre el perfil del usuario.

11. Diseño de Interfaces

11.1 Módulo de Autenticación y Registro (Login / Register)

Nombre: Pantalla de Acceso y Registro de Usuarios (LoginView / RegisterView)

Objetivo: Permitir al usuario ingresar de forma segura a su cuenta mediante credenciales validadas con JWT, o registrar una nueva microempresa y su respectivo administrador inicial.

Componentes:

- Campos de entrada de texto para correo electrónico y contraseña.
- Selector de rol y campos organizacionales para el registro inicial de la microempresa.
- Botones de acción principal ("Iniciar Sesión", "Registrarse").
- Enlace de recuperación de contraseña ("¿Olvidaste tu contraseña?").
- Indicador visual de carga (spinner) y alertas de error flotantes (toasts).

Validaciones:

- Validación del formato de correo electrónico válido mediante expresiones regulares (Regex).
- Control de campos obligatorios vacíos (bloqueo automático del botón de envío si faltan datos).
- Restricción de intentos fallidos (bloqueo temporal de 15 minutos tras 3 ingresos erróneos consecutivos).

Captura de pantalla: (Mockup conceptual: Interfaz limpia y minimalista centrada con el logotipo de Stock Velia, campos con bordes redondeados y tipografía moderna en tonos oscuros e institucionales).

Flujo de navegación:

- Inicio en pantalla de Login → Credenciales correctas → Redirección al Panel Principal (Dashboard).
- Si se selecciona registro → Pantalla de Registro de Microempresa → Registro exitoso → Redirección al Login.

11.2 Panel de Control Principal

Nombre: Panel General e Indicadores de Inventario (DashboardView)

Objetivo: Brindar al administrador y empleados una vista panorámica en tiempo real del estado de la microempresa, valorización de existencias y alertas críticas.

Componentes:

- Tarjetas de resumen métrico (Total de productos, stock disponible, valor económico total del inventario).
- Panel de alertas visuales prominentes para productos con stock bajo o agotado (resaltados en color rojo).
- Gráficos estadísticos de rotación de inventario y entradas/salidas recientes.
- Barra de navegación lateral fija con accesos directos a Catálogo, Movimientos y Reportes.

Validaciones:

- Verificación de permisos de usuario activos (si el rol no tiene privilegios, se ocultan secciones administrativas).
- Sincronización automática de datos mediante WebSockets para refrescar las métricas sin recargar el navegador.

Captura de pantalla: (Mockup conceptual: Diseño tipo grid responsivo, con tarjetas superiores de indicadores numéricos y gráficos circulares/de barras organizados junto a una lista de alertas críticas en tiempo real).

Flujo de navegación:

- Acceso desde el Login → Visualización del Dashboard → Clic en alertas o menú lateral para navegar hacia el Módulo de Productos o Movimientos.

11.3 Gestión de Catalogo de Productos

Nombre: Módulo de Administración de Catálogo (CatalogView)

Objetivo: Permitir registrar, buscar, editar y dar de baja de forma lógica los productos del inventario de la microempresa.

Componentes:

- Barra de búsqueda en tiempo real por nombre o código de barras.
- Filtros desplegados por categoría y estado de stock.
- Tabla interactiva de productos (Código, Nombre, Categoría, Precio, Stock Actual, Stock Mínimo, Acciones).
- Botones de acción rápida ("Nuevo Producto", "Editar", "Borrado Lógico").
- Modal emergente con formularios de alta y modificación de atributos.

Validaciones:

- Restricción de valores negativos en los campos de precio, stock y stock mínimo (≥ 0).
- Verificación de unicidad en el código de barras para evitar duplicados en la base de datos.
- Confirmación obligatoria mediante cuadro de diálogo antes de ejecutar un borrado lógico.

Captura de pantalla: (Mockup conceptual: Tabla de datos estructurada con paginación inferior, botones de iconos claros para edición y una barra superior con filtros avanzados y botón de alta destacado).

Flujo de navegación:

- Menú principal → Clic en "Catálogo" → Visualización de tabla → Opción de registrar nuevo producto (abre modal) o editar producto existente → Guardado y actualización automática de la vista.

11.4 Registro de Movimientos de Inventario

Nombre: Módulo de Entradas y Salidas (InventoryMovementsView)

Objetivo: Registrar de manera controlada el ingreso de mercancía (compras/devoluciones) y la salida de productos (ventas/daños), actualizando el stock de forma automática.

Componentes:

- Selector de tipo de movimiento (Entrada / Salida / Baja).
- Buscador y selector de productos activos.
- Campo numérico de cantidad y selector de motivo específico (Compra, Venta, Devolución, Daño).
- Campo de observaciones o notas adicionales.
- Botón de confirmación de transacciones y tabla de historial reciente de movimientos.

Validaciones:

- Validación obligatoria de que la cantidad a retirar no supere el stock actual disponible (bloqueo de transacción y alerta de "Stock Insuficiente").
- Validación de que la cantidad ingresada sea estrictamente mayor a cero (> 0).
- Captura automática del ID del usuario responsable de la sesión actual.

Captura de pantalla: *(Mockup conceptual: Formulario dividido en dos secciones; la izquierda para el ingreso de datos del movimiento con desplegables limpios, y la derecha con una vista previa del producto seleccionado y su stock actual).*

Flujo de navegación:

- Menú principal → Clic en "Movimientos" → Selección del tipo de operación → Llenado del formulario y procesamiento → Emisión de alerta de éxito y propagación del nuevo stock en tiempo real a las demás vistas.

12. Prototipos

12.1 Enfoque y Alcance del Prototipado

La fase de prototipado de Stock Velia tiene como objetivo materializar los requerimientos funcionales y las historias de usuario en maquetas visuales interactivas. Esto permite validar la experiencia de usuario (UX/UI), el flujo de navegación entre módulos y la ergonomía de las pantallas antes de iniciar con la codificación formal en el frontend.

12.2 Prototipos de Baja Fidelidad (Wireframes Estructurales)

- **Propósito:** Definir la distribución espacial (layout), la jerarquía de la información y la ubicación de los componentes críticos en los distintos dispositivos (enfocándose principalmente en pantallas de escritorio y portátiles para la gestión operativa).
- **Representación Clave:**
 - **Pantalla de Login / Autenticación:** Estructura limpia centrada con el formulario de credenciales y el logotipo institucional.
 - **Dashboard Principal:** Diseño en cuadrícula (grid) que prioriza los indicadores numéricos superiores y las alertas visuales de stock bajo.
 - **Módulo de Catálogo y Movimientos:** Tablas estructuradas con barras de búsqueda superiores, filtros laterales y modales de confirmación para evitar acciones accidentales de borrado o salidas con stock insuficiente.

12.3 Prototipos de Alta Fidelidad (Mockups Interactivos)

- **Propósito:** Simular el comportamiento real de la aplicación aplicando la guía de estilos visuales, paleta de colores corporativa (tonos profesionales, oscuros e institucionales combinados con alertas en rojo y verde de éxito), tipografías y estados interactivos de botones y menús.
- **Herramientas de Diseño Recomendadas:**
 - **Figma**, como herramienta central para el diseño colaborativo de los mockups y la definición de componentes reutilizables.
- **Interactividad Clave Validada:**
 - Simulación del flujo de inicio de sesión con validación visual de errores de credenciales.
 - Comportamiento dinámico de los modales de registro de productos y movimientos de inventario.
 - Visualización del cambio de estado visual (alertas en rojo) cuando un producto alcanza o desciende del umbral de stock mínimo definido.

12.4 Validación con Stakeholders y Usuarios

- **Proceso de Retroalimentación:** Los prototipos de alta fidelidad se emplean para realizar pruebas de usabilidad con los futuros usuarios (administradores de microempresas y empleados operativos), asegurando que la curva de aprendizaje sea mínima y que el acceso a las funciones críticas de inventario en tiempo real se realice en menos de tres clics.

13. Estructura del Proyecto

La organización del código fuente de Stock Velia se divide en tres niveles principales: el servidor (Backend), la interfaz de usuario (Frontend) y los archivos raíz del repositorio. La arquitectura del backend implementa el patrón MVC (Modelo-Vista-Controlador) adaptado a servicios RESTful con Node.js, mientras que el frontend utiliza una arquitectura modular basada en componentes y vistas con React.

Descripción Detallada de Módulos Clave:

- **backend/src/config/:** Aloja las conexiones críticas, destacando db.js para la persistencia en PostgreSQL y mailer.js para la automatización de correos electrónicos corporativos.
- **backend/src/controllers/:** Contiene los módulos de control (authController, productoController, movimientoController, etc.) que procesan las reglas de negocio y responden a las peticiones del cliente.
- **frontend/src/components/Dashboard/:** Agrupa los componentes internos que componen las pestañas operativas del sistema (gestión de stock, categorías, usuarios y reportes).
- **frontend/src/services/:** Centraliza la comunicación asíncrona hacia el backend mediante servicios independientes basados en Axios (api.js), facilitando la reutilización del código en las distintas vistas de la aplicación.

14. Tecnologías Utilizadas

Para el desarrollo e implementación del sistema Stock Velia, se ha seleccionado un stack tecnológico moderno, robusto y eficiente basado en JavaScript tanto para el servidor como para el cliente, garantizando una alta velocidad de respuesta, sincronización en tiempo real y escalabilidad para PyMEs.

14.1 Arquitectura y Lenguajes Base

- **JavaScript (ES6+):** Lenguaje de programación principal empleado en toda la solución (tanto en el backend con Node.js como en el frontend con React).
- **Node.js:** Entorno de ejecución de JavaScript del lado del servidor, optimizado para manejar operaciones asíncronas y solicitudes simultáneas mediante un modelo de E/S sin bloqueo.

14.2 Backend (Servidor y API REST)

- **Express.js:** Framework minimalista y flexible para Node.js, utilizado para la construcción de la API REST, estructuración de rutas corporativas y gestión de middleware.
- **JSON Web Token (JWT):** Estándar abierto para la autenticación y control de sesiones de usuario de forma segura y sin estado (stateless).
- **Nodemailer:** Módulo para Node.js implementado para la automatización y envío de correos electrónicos transaccionales (como la recuperación de contraseñas mediante códigos de seguridad).
- **Bcrypt / Librerías de cifrado:** Utilizadas para el hashing seguro de contraseñas antes de su persistencia en la base de datos.

14.3 Frontend (Interfaz de Usuario)

- **React:** Biblioteca de JavaScript basada en componentes para la construcción de interfaces de usuario dinámicas, modulares y de alto rendimiento.
- **React Router:** Enrutador oficial para la gestión de navegación entre vistas (pages) y protección de rutas privadas (Dashboard, módulos internos).
- **Axios:** Cliente HTTP basado en promesas para realizar las peticiones asíncronas entre la SPA de React y los endpoints del backend.
- **CSS3:** Hojas de estilo modularizadas por componentes y vistas para lograr un diseño responsivo, adaptado a distintos dispositivos y con una interfaz visual limpia.

14.4 Base de Datos y Persistencia

- **PostgreSQL:** Sistema de gestión de bases de datos relacional robusto y ACID, encargado de garantizar la integridad referencial del inventario, usuarios, categorías y movimientos.
- **Pool de conexiones (pg):** Conector optimizado para la comunicación eficiente entre el servidor Node.js y el motor relacional PostgreSQL.

14.5 Control de Versiones y Herramientas de Desarrollo

- **Git & GitHub:** Sistema de control de versiones distribuido para la gestión de ramas, resolución de conflictos en equipo y almacenamiento del repositorio remoto.
- **Visual Studio Code (VS Code):** Entorno de desarrollo integrado (IDE) principal, complementado con extensiones de depuración y formateo de código.

15. APIs

El sistema expone una arquitectura de servicios RESTful protegida mediante tokens de autenticación JWT (JSON Web Tokens) y mecanismos de control de sesiones revocables (*Blacklist*). A continuación se detallan los endpoints agrupados por módulos de negocio:

15.1 Módulo de Autenticación (/api/auth)

A. Registro de Empresa y Usuario Administrador

Método HTTP: POST

URL: /api/auth/register

Descripción: Registra de forma simultánea una nueva organización empresarial y su usuario administrador inicial, encriptando la contraseña mediante bcrypt.

Parámetros: Ninguno (Público).

Request (Body):

JSON

```
{
  "nombre_empresa": "Comercializadora Velia",
  "nombre_usuario": "Ana",
  "apellido_usuario": "María",
  "email": "maria3465@correo.com",
  "password": "Password123"
}
```

Response (201 Created):

JSON

```
{
  "mensaje": "Empresa y Usuario Administrador registrados con éxito.",
  "usuario": {
    "id_usuario": 1,
    "nombre": "Ana Maria",
    "correo": "maria3465@correo.com"
  }
}
```

Códigos de Error:

- **400:** El correo del usuario ya está registrado.
- **500:** Error interno del servidor al registrar.

B. Inicio de Sesión (Login)

Método HTTP: POST

URL: /api/auth/login

Descripción: Autentica las credenciales del usuario, validando estados de cuenta, bloqueos temporales por intentos fallidos y generando un token JWT con vigencia de 2 horas.

Parámetros: Ninguno (Público).

Request (Body):

JSON

```
{
  "correo": "maria3465@correo.com",
  "password": "Password123"
}
```

Response (200 OK):

JSON

```
{
  "mensaje": "Inicio de sesión exitoso.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cGU6IjYiLCJ1b2R0IjoiYXNpLn...",
  "usuario": {
    "id_usuario": 1,
    "id_empresa": 1,
    "nombre": "Ana María",
    "correo": "maria3465@correo.com",
    "id_rol": 1,
    "nombre_empresa": "Comercializadora Velia"
  }
}
```

Códigos de Error:

- **404:** El correo electrónico no está registrado.
- **403:** Cuenta desactivada.
- **423:** Cuenta bloqueada temporalmente (por exceder 5 intentos fallidos).
- **401:** Contraseña incorrecta (devuelve los intentos restantes).
- **500:** Error interno del servidor al iniciar sesión.

C. Solicitud de Recuperación de Contraseña

Método HTTP: POST

URL: /api/auth/forgot-password

Descripción: Genera un código aleatorio de 6 dígitos con vigencia de 15 minutos y lo envía al correo electrónico del usuario mediante Nodemailer.

Parámetros: Ninguno (Público).

Request (Body):

JSON

```
{
  "correo": "maria3465@correo.com"
}
```

Response (200 OK):

JSON

```
{
  "mensaje": "Código de verificación enviado al correo electrónico."
}
```

15.7 Códigos de Error:

- **404:** El correo electrónico no está registrado.
- **500:** Error al enviar el código de recuperación.

D. Restablecer Contraseña

Método HTTP: POST

URL: /api/auth/reset-password

Descripción: Valida el código de verificación de 6 dígitos recibido por correo y actualiza la contraseña del usuario tras aplicar un nuevo hash con bcrypt.

Parámetros: Ninguno (Público).

Request (Body):

JSON

```
{
  "correo": "maria3465@correo.com",
  "codigo": "485920",
  "nuevaPassword": "NewPassword123"
}
```

Response (200 OK):

JSON

```
{
  "mensaje": "Contraseña restablecida con éxito."
}
```

Códigos de Error:

- **400:** El código de verificación es incorrecto o ha expirado.
- **500:** Error interno del servidor al cambiar la contraseña.

E. Cierre de Sesión (Logout)

Método HTTP: POST

URL: /api/auth/logout

Descripción: Invalida el token actual registrándolo en una tabla de lista negra (*Blacklist*) para evitar futuros usos.

Parámetros: Header Authorization: Bearer <JWT>

Request (Body): Ninguno.

Response (200 OK):

JSON

```
{
  "mensaje": "Sesión cerrada con éxito. Token revocado."
}
```

Códigos de Error:

- **401:** Acceso denegado (No se proporcionó token o el token ya fue revocado).
- **403:** Token inválido o expirado.
- **500:** Error al cerrar sesión.

15.2 Módulo de Productos (/api/productos)

A. Crear Producto

Método HTTP: POST

URL: /api/productos

Descripción: Registra un nuevo artículo en el inventario asociado a la empresa del usuario autenticado, verificando que el código/SKU no esté duplicado.

Parámetros: Header Authorization: Bearer <JWT>

Request (Body):

JSON

```
{
  "id_categoria": 2,
  "codigo": "SKU-998877",
  "nombre": "Taladro Percutor 500W",
  "descripcion": "Herramienta eléctrica de uso industrial",
  "precio": 185000.00,
  "stock": 15,
  "stock_minimo": 3
}
```

Response (201 Created):

JSON

```
{
  "mensaje": "Producto registrado con éxito.",
  "producto": {
    "id_producto": 12,
    "id_empresa": 1,
    "id_categoria": 2,
    "codigo": "SKU-998877",
    "nombre": "Taladro Percutor 500W",
    "precio": "185000.00",
    "stock": 15,
    "stock_minimo": 3,
    "estado": true
  }
}
```

Códigos de Error:

- **400:** El código de barras o SKU ya está registrado en tu empresa.
- **401 / 403:** Errores de autenticación o token inválido.
- **500:** Error al registrar el producto.

B. Obtener Productos Activos

Método HTTP: GET

URL: /api/productos

Descripción: Retorna el listado completo de productos activos (estado = TRUE) vinculados a la empresa del usuario, incluyendo su categoría asociada.

Parámetros: Header Authorization: Bearer <JWT>

Request (Body): Ninguno.

Response (200 OK):

JSON

```
[
  {
    "id_producto": 12,
    "nombre": "Taladro Percutor 500W",
    "codigo": "SKU-998877",
    "precio": "185000.00",
    "stock": 15,
    "nombre_categoria": "Herramientas"
  }
]
```

Códigos de Error:

- **500:** Error al obtener los productos.

C. Actualizar Producto

Método HTTP: PUT

URL: /api/productos/:id

Parámetros:

- **URL Params:** id (Identificador numérico del producto).
- **Header:** Authorization: Bearer <JWT>

Request (Body):

JSON

```
{
  "id_categoria": 2,
  "codigo": "SKU-998877",
  "nombre": "Taladro Percutor 600W Pro",
  "descripcion": "Versión actualizada con mayor potencia",
  "precio": 195000.00,
  "stock_minimo": 5
}
```

Response (200 OK):

JSON

```
{
  "mensaje": "Producto actualizado con éxito.",
  "producto": { "id_producto": 12, "nombre": "Taladro Percutor 600W Pro" }
```

```
}
```

Códigos de Error:

- **404:** Producto no encontrado o no pertenece a tu empresa.
- **500:** Error al actualizar el producto.

D. Eliminación Lógica de Producto

Método HTTP: DELETE

URL: /api/productos/:id

Descripción: Realiza un borrado lógico cambiando el estado del producto a falso (estado = FALSE).

Parámetros:

- **URL Params:** id
- **Header:** Authorization: Bearer <JWT>

Request (Body): Ninguno.

Response (200 OK):

JSON

```
{
  "mensaje": "Producto desactivado del inventario con éxito."
}
```

Códigos de Error:

- **404:** Producto no encontrado.
- **500:** Error al eliminar el producto.

15.3 Módulo de Categorías (/api/categorias)

Método HTTP / URL:

- POST /api/categorias (Crear categoría).
- GET /api/categorias (Listar categorías ordenadas alfabéticamente).

Descripción: Gestión de las agrupaciones clasificatorias para los productos de inventario.

Parámetros: Header Authorization: Bearer <JWT>

Request (Body - POST):

JSON

```
{
  "nombre": "Herramientas",
  "descripcion": "Herramientas de ferretería y construcción"
}
```

Response (201 Created / 200 OK): Devuelve el objeto de la categoría creada o un arreglo con el listado completo.

Códigos de Error:

- **400:** La categoría ya existe en el sistema.
- **500:** Error al crear u obtener las categorías.

15.4 Módulo de Movimientos de Inventario (/api/movimientos)

Método HTTP / URL:

- POST /api/movimientos (Registrar un movimiento de entrada/salida de stock).
- GET /api/movimientos (Obtener el historial completo de movimientos asociados a la empresa).

Descripción: Control transaccional de existencias por producto, aplicando validaciones de pertenencia empresarial y restricciones automáticas por stock insuficiente.

Parámetros: Header Authorization: Bearer <JWT>

Request (Body - POST):

JSON

```
{
  "id_producto": 12,
  "tipo_movimiento": "SALIDA",
  "motivo": "VENTA",
  "cantidad": 2,
  "observacion": "Factura de venta #0012"
}
```

Response (201 Created):

JSON

```
{
  "mensaje": "Movimiento de inventario procesado con éxito.",
  "movimiento": { "id_movimiento": 45, "tipo_movimiento": "SALIDA", "cantidad": 2 }
}
```

Códigos de Error:

- **403:** No tienes autorización para alterar el stock de este producto.
- **400:** Stock insuficiente (capturado desde la base de datos).
- **500:** Error al procesar el movimiento de inventario u obtener el historial.

Módulo de Usuarios (/api/usuarios)

Método HTTP / URL:

- GET /api/usuarios?id_empresa=1 (Listar usuarios de una empresa).
- POST /api/usuarios (Crear un usuario secundario con validación estricta de contraseña).
- PUT /api/usuarios/estado/:id (Alternar el estado activo/inactivo de un usuario).

Descripción: Administración interna del personal operativo de la empresa.

Request (Body - POST):

JSON

```
{  
  "nombre": "Carlos Pérez",  
  "correo": "carlos@correo.com",  
  "password": "SecurePassword1",  
  "id_empresa": 1  
}
```

Response (201 Created / 200 OK): Retorna los datos del usuario modificado o creado.

Códigos de Error:

- **400:** Todos los campos son obligatorios / La contraseña no cumple con los requisitos de seguridad (mínimo 6 caracteres, una mayúscula, una minúscula y un número) / El correo ya está registrado.
- **404:** Usuario no encontrado o no se puede modificar (protección sobre roles administrativos principales).
- **500:** Error interno al gestionar usuarios.

16. Seguridad

16.1 Autenticación

16.2 Autorización

16.3 Roles

16.4 Hash de contraseñas

16.5 Protección SQL Injection

16.6 Protección XSS

16.7 Protección CSRF

16.8 Validaciones backend

16.9 Validaciones frontend

17. Rendimiento

17.1 Optimización de consultas

17.2 Índices

17.3 Caché

17.4 Consumo de memoria

17.5 Consumo de CPU

18. Pruebas

18.1 Plan de pruebas

18.2 Casos de prueba

18.3 Pruebas funcionales

18.4 Pruebas de integración

18.5 Pruebas de rendimiento

18.6 Pruebas de seguridad

18.7 Evidencias

18.8 Resultado esperado

18.9 Resultado obtenido

18.10 Estado

19. Gestión del Proyecto

19.1 Metodología

19.1.1 Scrum

19.1.2 Kanban

19.1.3 Cascada

19.2 Cronograma

19.3 Sprint

19.4 Actividades

19.5 Entregables

19.6 Roles

20. Control de Versiones

20.1 Estrategia Git

20.2 Ramas

20.3 Commits

20.4 Releases

- 21. Instalación
 - 21.1 Requisitos previos
 - 21.2 Dependencias
 - 21.3 Configuración
 - 21.4 Variables de entorno
 - 21.5 Instalación
 - 21.6 Ejecución
 - 21.7 Despliegue

22. Manual Técnico

23. Manual de Usuario

24. Mantenimiento

25. Riesgos

26. Futuras Mejoras

27. Conclusiones

28. Recomendaciones

29. Bibliografía